

谢冠文周报

2026年4月16日

本周是week8，主要目标是继续深入归因，并且开始引入了传感器噪声模型，进一步调研和多机搜索有关的文献。

DORA: Distributed Online Risk-Aware Explorer (ICRA 2022)

本论文解决的是多无人机探索协调的问题。现有方法依赖中央服务器构建集体信念地图，这需要完美的连接和高带宽，存在通信瓶颈和单点故障风险，的分布式探索策略（例如Voronoi图或前沿点FBE）关注最大化覆盖范围，缺失风险感知。本论文关注去中心化、风险感知的多机器人探索算法，仅依靠局部计算和邻居通信，既能最大化覆盖，又能最小化机器人因暴露于危险而失效的概率。

为了解决通信瓶颈，DORA 使用 **虚拟 stigmergy (Virtual Stigmergy)** 技术

1. **风险信念地图 (r-map)**: 存储每个网格单元被感知到的风险水平。
2. **信息时效地图 (e-map)**: 存储每个网格单元最后被访问的时间。

机器人只读写其邻近单元格的数据。当多个机器人写入同一键值时，采用平均值合并或保留最高ID数据。这种方法通信成本低，且对临时断连具有鲁棒性。

机器人通过局部梯度下降法决定下一步移动方向，旨在同时**最小化风险和最大化信息增益**。

- **风险梯度 (∇r_i)**: 指向风险降低的方向（基于局部邻域的风险差值）。
- **探索梯度 (∇e)**: 指向长时间未访问或未访问区域的方向（基于时间差 Δt ）。
- 运动向量综合了风险、探索和避障，使得其只依赖于局部邻域信息,且计算量也只依赖于邻域大小。

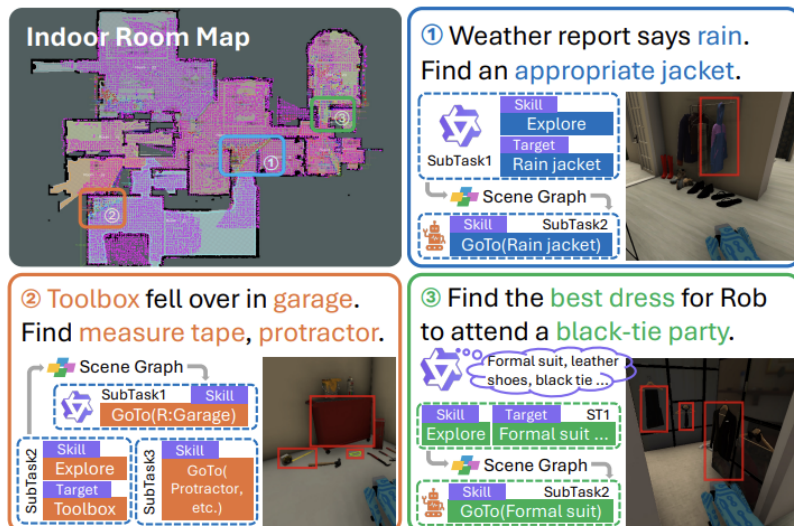
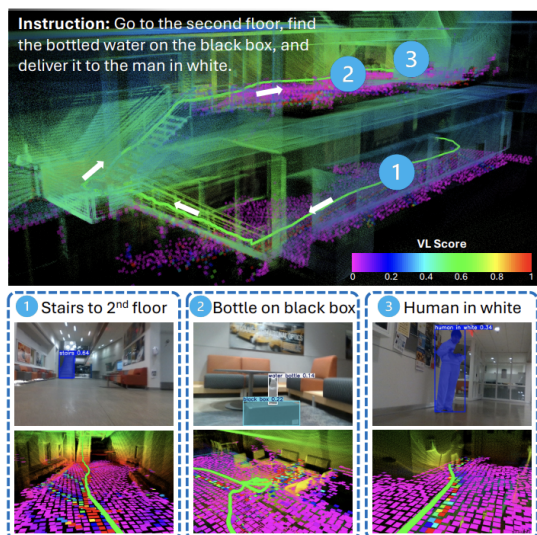
实验表明：20个机器人下机器人仍然活跃，通信量很小。这归根到底是中心化、局部信息的风险感知探索，通过牺牲短期的高风险区域覆盖，换取了长期的团队存活率和总体探索效率。

VL-Nav: A Neuro-Symbolic Approach for Reasoning-based Vision-Language Navigation (CMU 2025)

提出一种神经符号系统，将神经网络的语义理解能力与符号系统的精确引导相结合，解决两个关键能力缺失：(1) 准确的推理以解析抽象多目标指令；(2) 高效的探索策略以在大规模环境中快速定位多个目标。

- **技术突破**: 提出 NeSy（神经符号）架构。将 VLM 的语义理解力（如“寻找红色帐篷”）与 3D 场景图（的符号化记忆结合）。

- 架构价值：其 IBTP（基于实例的目标点生成）模仿了人类“先发现疑似点、再靠近确认”的搜索逻辑。该框架为后续接入 VLM 提供了标准的 API 接口范本，即“大模型做逻辑分解，几何规划器做安全落地”。



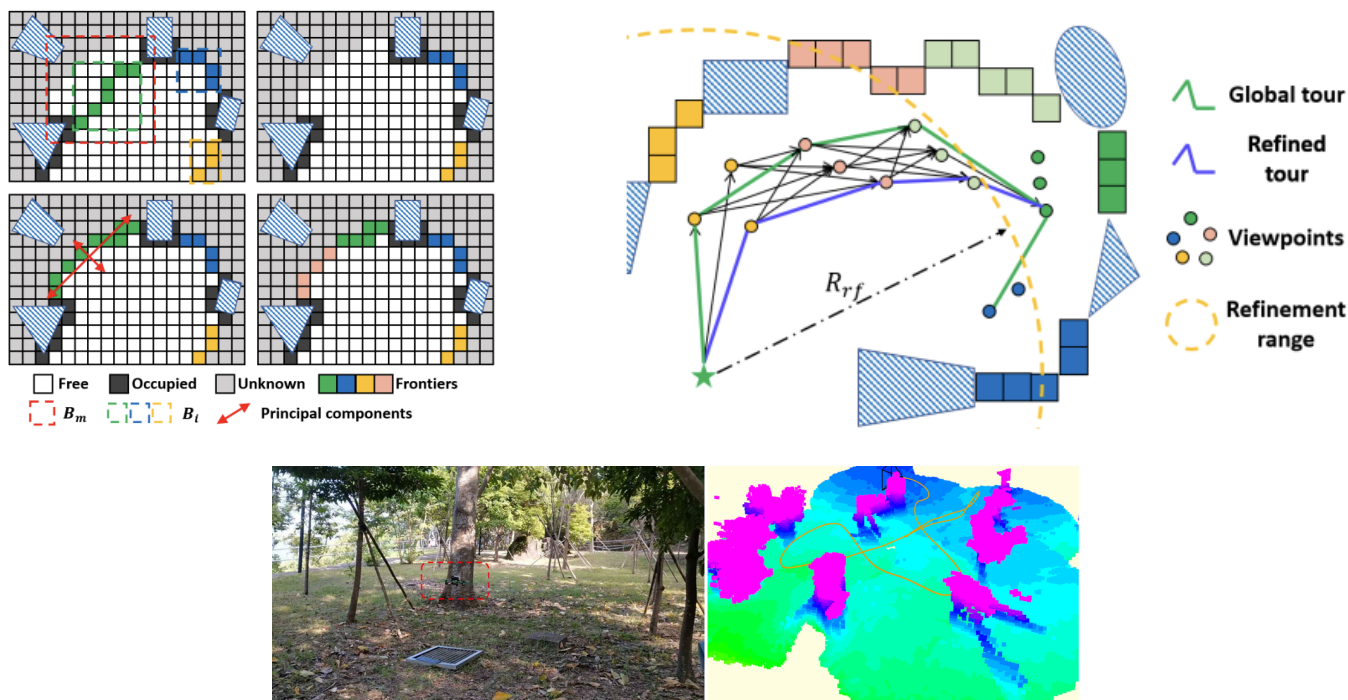
FUEL: Fast UAV Exploration using Incremental Frontier Structure and Hierarchical Planning (Shaojie Shen, RAL 2021)

本论文提出一种名为 FUEL 的分层框架，旨在复杂未知环境中实现快速的无人机自主探索，解决上述效率、敏捷性和实时性问题。

- 核心贡献：提出了从粗到细的三级规划闭环。利用 ATSP（非对称旅行商问题）求解全局路径，再通过局部视点细化与最小时间 B-样条生成物理轨迹。
- 科研启示：FUEL 证明了“运动一致性”对提高搜索效率的重要性，即减少不必要的掉头和震荡，这与我们目前提升 10m/s 航速、压低 Jerk 积分的目标高度契合。

B. 真实世界实验

- 平台：搭载 Intel RealSense D435i 相机和 Intel Core i7 CPU 的定制四旋翼无人机。
- 结果：
 - 无人机能够在复杂环境中进行敏捷的自主探索。
 - 系统能够实时处理传感器数据，更新地图，并生成避开障碍物的最小时间轨迹。
 - 证明了该方法在真实物理约束和感知噪声下的鲁棒性。



RACER: Rapid Collaborative Exploration with a Decentralized Multi-UAV System (Shaojie Shen, TRO 2023)

本文旨在解决多无人机系统在通信受限环境下的高效、去中心化协同探索问题。RACer 系统由三个核心模块组成：基于 hgrid 的成对交互协调、基于 CVRP 的工作负载划分、以及分层探索规划。

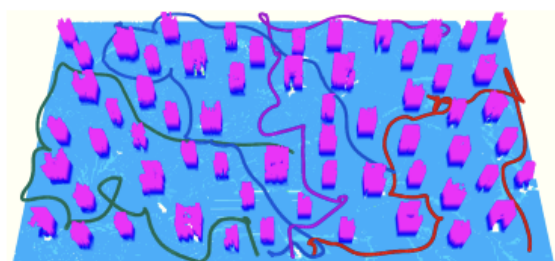
定义了多机协同的“版本答案”。通过 Hgrid 空间分解将未知环境单元化，并引入 CVRP（带容量限制的车辆路径问题）解决工作量平衡。其去中心化的成对交互机制，确保了系统在弱通信环境下的强鲁棒性。

系统实现细节

- 状态估计：使用 Omni-Swarm 进行去中心化的视觉-惯性-UWB 融合定位，建立统一坐标系。
- 地图共享：采用 UDP 广播新观测的体素块，并通过记账机制处理丢包，实现鲁棒的地图同步。

真实世界实验

- 平台：自定义四旋翼无人机，搭载 Jetson TX2、Realsense D435 相机和 UWB。
- 场景：
 - 室内杂乱环境**：2 架无人机在 10x6x2m 空间内，23秒完成探索。
 - 大型室内环境**：3 架无人机在 15x9x2m 空间内，33秒完成探索。展示了在线建图和轨迹规划的实时性。
 - 室外森林环境**：3 架无人机在 15x12x2m 自然环境中，33秒完成探索。验证了在非结构化环境中的鲁棒性。
- 成就：这是首次在真实世界中实现完全去中心化、无外部定位辅助、无中央服务器的多无人机自主协同探索。



论文小结

综上所述，当前学术界的研究趋势呈现出三个核心特征：

1. 去中心化与轻量化：如 RACER 和 DORA，通过局部信息的局部交换替代全局地图的强同步，以应对复杂环境的通信瓶颈。
2. 分层架构的必然性：无论是 FUEL 的从粗到细，还是 VL-Nav 的神经-符号结合，都采用了“高层决策指导、底层几何避障”的解耦思路。
3. 从“全覆盖”到“信息增益”：现代规划器不再盲目扫描，而是计算“信息增益”或“风险权重”，使搜索行为更加拟人化。

【找到物体，搜寻】

参考实施路径

响应老师关于“快速、完整遍历未知空间”的指示，为本课题定制了三阶段演进路线：

Phase 1: 单机高效感知与前沿点管理

- **参考逻辑**：基于 FUEL 的分层思想，将 A* 的搜索目标从几何点提升为“前沿点簇”。
- **实施重点**：在现有的 Python 调度层引入信息增益计算。不仅评估“路径短”，更评估“扫过的未知体素多”。
- **目标**：实现单机在 5-10m/s 高速下，依然能保持对未知边界的连续追踪，不出现“反复折返”。

Phase 2: 去中心化协同与负载均衡

- **核心逻辑**：借鉴 RACER 的成对交互机制。
- **实施重点**：
 - **空间分解**：利用 Hgrid 或 Voronoi 图将室内场景进行拓扑切分。
 - **任务竞价**：实现一个轻量级的基于 CVRP（带容量限制的车辆路径问题）的分配器。
- **目标**：当三机相遇时，通过交换高度压缩的 `Bounding Box` 认知，自动划定各自负责的搜索扇区，实现 $1+1+1 > 3$ 的覆盖效率。

Phase 3: 弱通信下的语义一致性建图

- **核心逻辑**：参考 DORA 的风险感知与分布式信念图。

- **实施重点：**在 `swarm_bridge` 中加入差异化地图同步。各机不再强行同步全局点云，而是只同步“拓扑关键点”与“已覆盖面积指纹”。
- **目标：**即便在密林或室内信号突发衰减时，集群仍能通过离线推演保持搜索任务的逻辑完整。

10m/s最终问题确认：地图太小,感知深度不足

本周通过对 10m/s 失败样本的深度审计，我发现此前对系统不稳定的归因存在偏差，并找到了真正的物理破解点。

- **时序优化结果：**

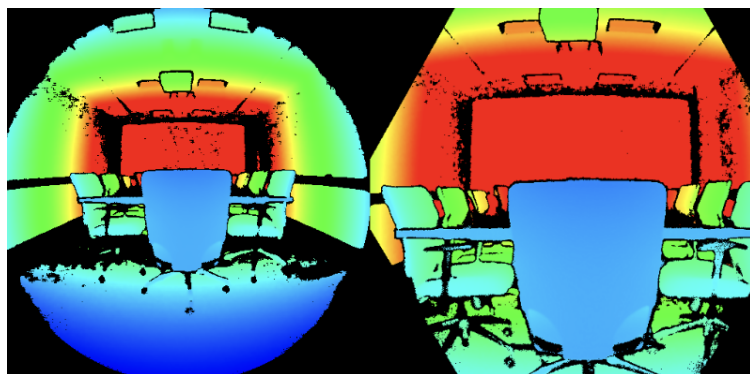
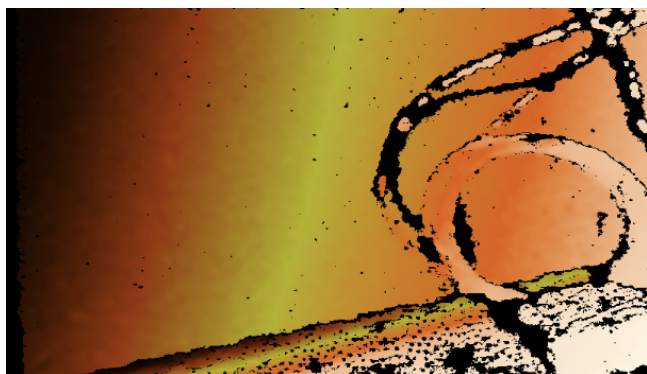
通过对控制指令积压和规划器启动时序的校准，已成功将系统指令闭环延迟稳定在 60ms 左右。实验证明，过分提高规划视野虽能提升规划时延宽容度，但会因计算开销激增而反向推高控制时延，必须寻求动态平衡。

- **物理边界的重构（核心发现）：**

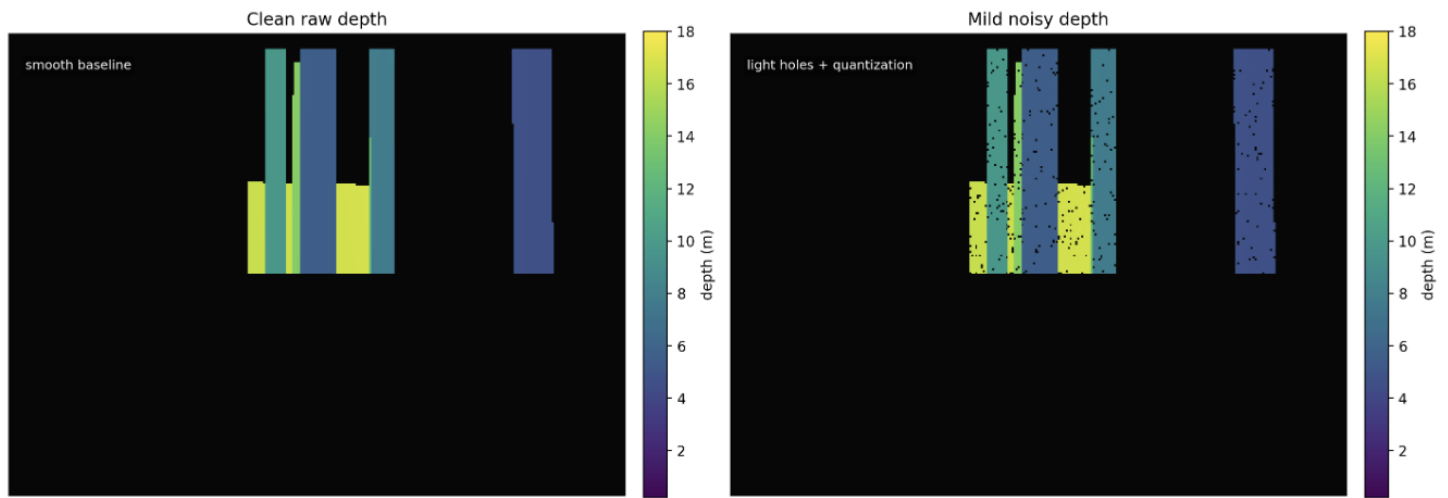
此前 Drone_2 频繁崩溃的根源不在于感知范围不足，而在于动力学参数设置过于保守，导致轨迹曲率无法收敛。

- 参数对标：此前设置的 $a=6.5\text{m/s}^2$ 、 $\text{jerk}=12\text{m/s}^3$ 难以支持 10m/s 下的紧急避障机动。
- 优化方案：参考多篇论文论文基准，将 a 提升至 12m/s^2 ，Jerk 提升至 15m/s^3 。
- 验证结论：在不盲目扩张 A^* 搜索范围的前提下，通过释放物理极限，本周已实现 8m/s 下的稳定穿梭。这证明了 Gazebo 在高动态下的数值稳定性足以支撑 10m/s 实验，关键在于算法输出的“刚性”是否匹配物理边界。

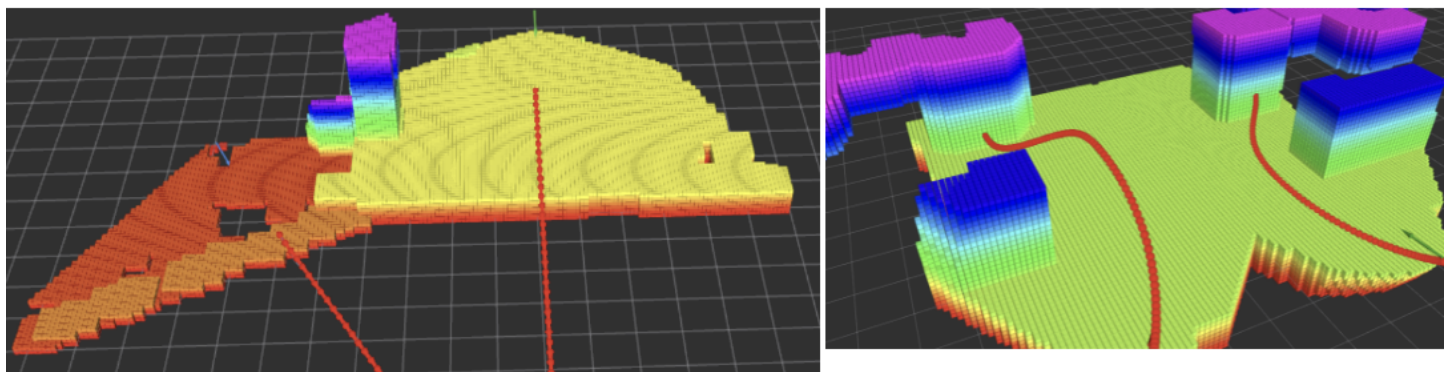
传感器噪声：从滤波下手



深度相机主要的噪声：毛刺表面、飞点与边缘伪影、空洞与缺失数据。下面是初步模拟的效果（包含空洞、距离增加劣化、量化台阶）：



这一步最主要的影响是会导致生成gridmap不干净，而这样的噪声会被障碍物膨胀的过程显著增强，加大噪声。



高速主要会导致下面的问题：

- 物体边缘出现像素错位，导致深度图边缘“拉丝”或直接产生无效值（NaN）=> 噪声注入器中加入“**动态边缘腐蚀**”逻辑
- 运动模糊诱导的距离偏差 => 拟合深度误差方差的速度平方线性关系
- 时间延迟 => 0.3-0.6m物理偏移，不过是有“方向性”的，主要是和控制延迟的非线性耦合 => 根据控制延迟进行方向补偿

被封印的代码库

本部分也对实物部署至关重要，具体来说，ego-planner-v2包含了大量的被封印的函数，或者功能：

```
// check consistency with last image, disabled...
if (false) {
    pt_reproj = last_camera_r_inv * (pt_world - md_.last_camera_pos_);
    double uu = pt_reproj.x() * mp_.fx_ / pt_reproj.z() + mp_.cx_;
    // ... 判断当前点是否在上一帧深度图的投影范围内
}
```

它通过重投影检查前后帧的深度一致性，能有效过滤掉 80% 的感知噪点，但是需要手动将false取消掉。

下面是其他主要的被封印的函数：

- 指定参数 `fsm/realworld_experiment`：需手动打开 `/traj_start_trigger`，也即遥控器或者rviz才能启动
- 虚拟墙：`grid_map/enable_virtual_wall`，`grid_map/virtual_ceil`，`grid_map/virtual_ground` 手动设定，可以有效防止撞墙（与以前的边界为双保险）
- 多拓扑路径：`manager/use_multitopology_trajs`，会同时优化多条线路。本部分可以大幅提升避障能力。

基本实验结果：安全余量显著下降，这会导致

如前所述，由于一般ego-planner会选择把规划器的性能“榨干”如果不对深度图进行滤波和偏移，根据前面的讨论，5m/s的实现已经接近达到极限（已经出现了多次安全违规），增加之后，下面是大致的结论：

- 对于普通噪声，即使只是5mm的偏差也会导致drone_2碰撞（余量不足），但是开启后，而2m/s以下由于余量更多，不易出现问题。
- 对于延迟和高速带来的问题，难以彻底消除，但修复之后，通过增强多拓扑搜索相结合，可以显著提高规划成功率，但是计算效率仍然要权衡。

下周建议：

下周将继续固化 10m/s 的底层稳定性，并正式启动分布式探索任务的逻辑层搭建：

【算法端】分布式任务分配器Demo：在 Python 调度层实现简单的空间切分逻辑，验证三机在互不干扰的前提下，空间扫描率是否能达到单机的 2.5 倍以上。

【感知端】前沿点提取模块开发：利用 `GridMap` 接口识别已知与未知空域的交界，并在 Rviz 中实现可视化引导。

【调研端】CPP 数学模型构建：推导涵盖多机 FoV 耦合、能量消耗与搜索完整性的路径覆盖数学描述。

【环境端】极端场景压力验证：在迷宫和直赛道场景下，通过手动“外挂”更长感知距离，进一步探测系统在 $a=12\text{m/s}^2$ 极限下的物理性能上限。

digest (TODO)

- 开源基准（对比） => 仿真重要，实际机器要有
- 场景要足够复杂（楼层） => 室内跨楼层 无人机很少（工程为主，科研算法偏少）
- 规划频率 深入分析

2026年4月9日

阶段目标：完成工程链路的深度解耦，开始转向 10m/s 级极速飞行性能优化，标志方向的转换。

文献调研：面向复杂/未知环境的高速运动规划系统

本周重点调研了学术界关于未知环境覆盖规划、高速避障以及语义空间表征的重点成果。调研核心结论显示：当物理规划内核（如 EGO-Planner）达到亚毫秒级响应后，系统的瓶颈已向“任务逻辑与轨迹的平滑映射”及“感知不确定性下的决策稳定性”转移。

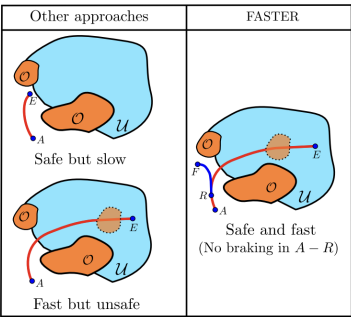
覆盖规划与采样逻辑的融合：C* (UCONN, TRO '26)

C* : A Coverage Path Planning Algorithm for Unknown Environments using Rapidly Covering Graphs

- **核心逻辑**：该工作利用快速覆盖图（Rapidly Covering Graphs）解决未知环境的完整性遍历问题，并结合 TSP 算法优化局部空洞。
- **科研启示**：它提供了一个将“离散逻辑任务”转化为“连续飞行轨迹”的成熟范本。
- **后续借鉴**：计划在 Python 调度层引入覆盖率感知模块。通过记录 FoV 在 3D 空间扫过的面积，将 A* 的搜索启发式从简单的“趋利性（向目标飞）”扩展为“探索性（向已知/未知交界处飞）”，为后续森林目标搜索奠定基础。

时空解耦与安全冗余策略：FASTER (MIT, TRO '21)

FASTER: Fast and Safe Trajectory Planner for Navigation in Unknown Environments



- **核心逻辑**：针对高速飞行中感知野有限的痛点，提出了“双轨迹并行”机制：一条在已知+未知空间追求极速的“完整轨迹”，一条在已知空间内保证刹停的“安全轨迹”。
- **关键洞察**：论文指出，在轨迹优化中，“**区间分配**”（即轨迹段落落在哪个凸多面体中) 对性能的影响远大于单纯的时间分配。
- **针对改进**：这指示了我们在提升速度至 10m/s 时，MINCO 优化器可能需要引入类似的**预测性安全约束**，而非仅仅依赖当前的软惩罚梯度。

极速飞行的 Sim2Real 最佳实践：UZH (SciRob '21)

Learning High-Speed Flight in the Wild

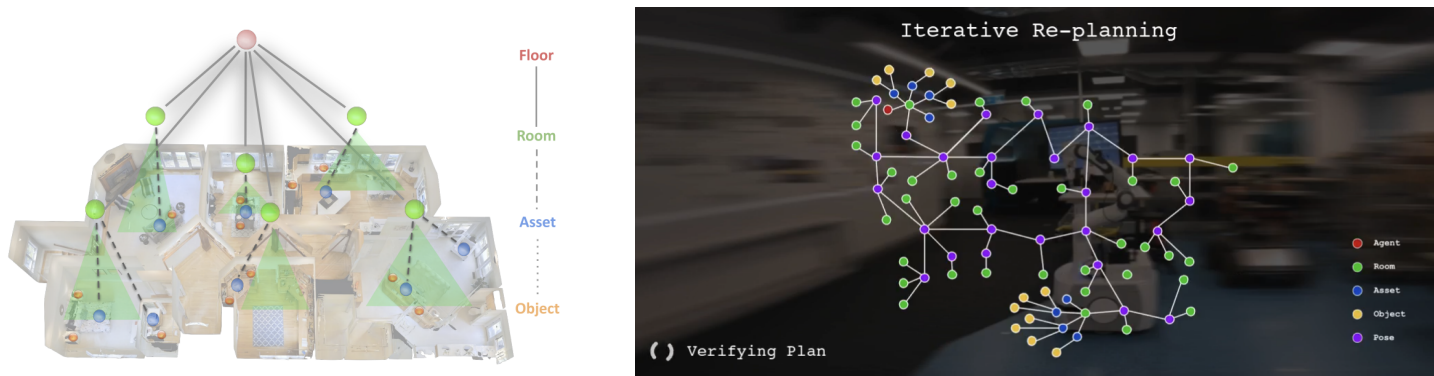
- **核心逻辑**：利用强化学习（RL）实现 10m/s 穿越森林。其成功关键在于通过 M-H 采样解决避障的**多模态性**（左绕或右绕），并利用 **Relaxed Winner-Takes-All Loss** 避免多模态解的均值崩溃。

- **工程价值：**该文定义了模拟传感器噪声（如深度图伪影）来训练鲁棒策略的方法，这对我们下阶段在 10m/s 场景下引入非理想感知模型具有直接指导意义。

语义空间的拓扑表征：SayPlan (QUT, CoRL '23)

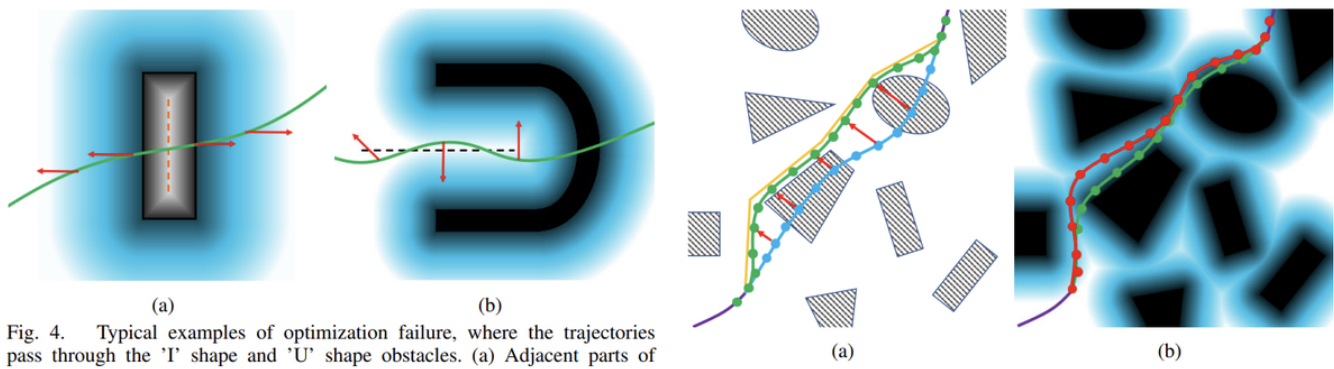
SayPlan: Grounding Large Language Models using 3D Scene Graphs for Scalable Robot Task Planning

- **核心逻辑：**提出了一种比传统 3D 场景图更“轻量”的层级化表征方法。利用视觉编码器（CLIP）提取物体特征，并通过 LLM 进行跨层级的任务推理。



主动感知与多路径搜索（RAPTOR, TRO '21）

RAPTOR: Robust and Perception-aware Trajectory Replanning for Quadrotor Fast Flight



高飞老师的这项工作通过拓扑路径搜索解决局部最优问题。针对 10m/s 场景，我们需要借鉴其“风险感知”思想，但必须在计算开销上做减法，以维持高频重规划。

文献调研小结

当前的科研共识正从“追求更快的单次规划”转向“追求更高维度的环境感知”。**多路径备份、拓扑引导、以及语义信息的引入**，是实现“类人机动”的关键。接下来的算法改进将围绕“如何保持轻量化的同时，引入多模态的搜索导向”展开。

基础设施建设：全自动化实验平台与洋葱式解耦

本周花费约 50% 的精力完成了工程层面的最后加固，旨在为接下来的大规模科研实验提供“确定性”的底座。

动化 Benchmark 体系：建立“数字化实验室”

- 实现功能：**开发了一套全流程闭环的基准测试体系。目前可实现多机集群一键拉起、数据全自动同步（rosbag/CSV/JSON）、多维科研指标（Jerk 积分、Replan 成功率、控制滞后）定量统计，并自动生成结构化 PDF 报告。
- 科研价值：**所有算法改进（如后续的异步规划）都将拥有**可回溯、可对比的证据链**，彻底告别“凭感觉调参”的非科学阶段。

“洋葱式”架构重构：软硬件部署的标准化

- 物理参数解耦：**通过将推力模型、PID 增益、避障权重等物理敏感参数彻底剥离至 YAML 配置文件，目前已实现“仿真与实机一套代码，一套 Launch 逻辑”。
- 迁移安全性：**该架构确保了在北京出差期间，远程开发的算法逻辑能够通过参数配置文件无感迁移至深圳的硬件平台，极大地降低了 Sim-to-Real 的转换成本与潜在风险。

基准测试：多梯度性能探测与边界识别

本周利用自研的自动化实验平台，针对EGO-Planner V2 (MINCO)开展了四个维度的压力测试，旨在量化时空联合优化内核在不同飞行速度与感知配置下的稳定性边界。

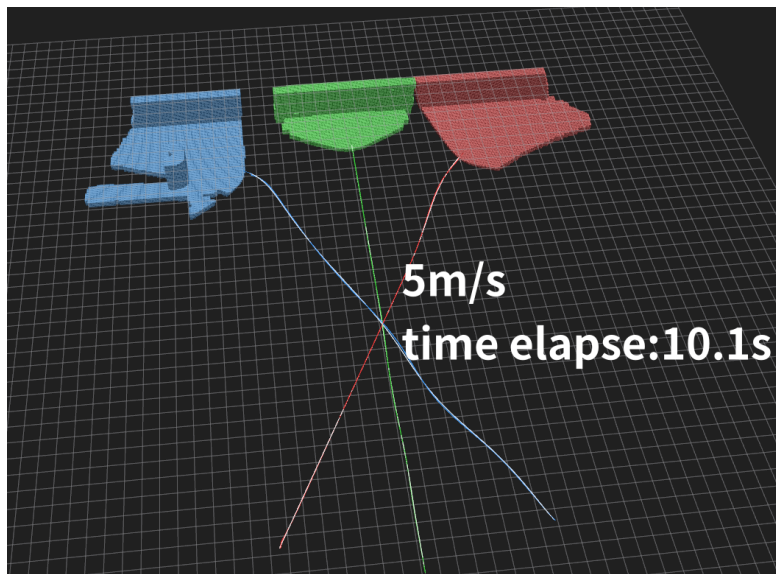
注意：感知层面从本来设置最小更新速度，转变为根据行进距离和安全事件，完成gridmap的生成事件触发。

速度梯度响应（Phase A）

在默认参数（Swarm_clearance=0.35m）下，探索系统在不同航速下的基础性能：

速度指令 (v_{max})	重规划成功率	最小安全间距	跟踪误差 (P95)	Jerk 积分	安全违规次数
1.2 m/s	94.12%	0.397 m	0.193 m	4.7	0
3.0 m/s	85.06%	0.316 m	0.541 m	11.9	7
5.0 m/s	100%	0.319 m	0.589 m	42.5	21

- 数据洞察：**在 1.2 m/s 档位观测到特定拓扑下的“病态样本”（Drone_2 触发紧急避障），这指明了低速稳健性仍受限于感知野与 A* 初值质量。
- 模式切换：**实验确认 Gazebo 可视化负载对 5m/s 以上的物理仿真存在显著干扰，后续 **10m/s 实验强制切换为 Headless（无界面）模式**以确保数据的一致性。



感知稳定性恢复（Phase B）

针对三机交汇时的频繁重规划与“决策拉扯”现象，测试了不同感知/安全权重的修复效果。在5m/s下实验，证明 `perception_clearance_plus` 是当前最优的平衡解：

- **优化方案：**感知野扩展至 $18 \times 18\text{m}$ ，`swarm_clearance` 提升至 0.5m。
- **结果：**重规划成功率提升至 **99.35%**，最小间距由 0.068m（Baseline）显著回升至 0.344m。
- **结论：**“提早、微小的规避”在分布式冲突解决中远优于“临近、剧烈的反应”。

时空权衡分析（Phase C）

在 5m/s 档位量化“飞行效率”与“控制品质”的博弈：

- **Time-Priority：**虽然到达时间最短，但跟踪误差（P95）激增至 18.8m，系统处于失控边缘。
- **Smooth-Priority：**虽然规划延迟略有上升，但安全违规数降低了 93%。
- **推荐基准：**中速档应优先采用 `smooth_priority` 配置，以换取控制系统的动态余量。

极限 10m/s 性能探测（Phase D）

这是本周最具科研挑战性的部分。通过参数消融实验，确立了 `d_rigid_clearance_balance` 为当前 10m/s 最佳候选参数集。

benchmark_default_vmax: 10.0

px4ctrl_kv: 2.8 # 速度环增益适当增加

swarm_clearance: 0.75 # 集群避碰间距（米），相比于10m/s继续增加

grid_map_obstacles_inflation: 0.3 # 障碍膨胀半径（米）

- **性能提升：**相比保守基线，其跟踪误差（P95）从 9.9m 压低至 **0.69m**，标志着 10m/s 档位从“仅理论可达”进入“具备工程实用价值”阶段。
- **算力瓶颈定量分析：**监控显示，10m/s 下规划栈的 CPU 占用呈现非线性激增（Ryzen 9 7945HX 16C32T）：

指标	1.2 m/s 基准	10.0 m/s 极限	增幅
Planner 栈 CPU 均值	0.85 Core	2.72 Core	320%
Planner 栈 CPU 峰值	2.77 Core	3.95 Core	触及 4 核物理上限
系统时钟频率均值	2183 MHz	2332 MHz	频率响应加速

系统异常诊断与可靠性修复

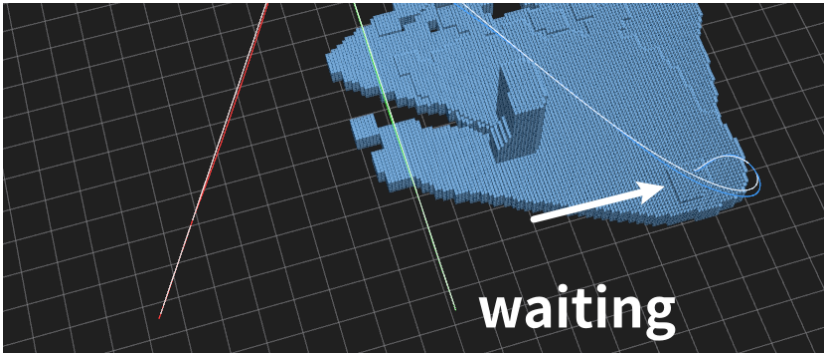
解决 10m/s 下的“控制滞后”幻象

- **现象：**基准测试记录到异常的 7600ms 控制滞后，但物理上并未炸机。
- **诊断结论：**这并非真实的控制延迟，而是“**计算算力坍塌导致的仿真时钟偏离**”。在 10m/s 高频重规划下，ROS 1 单线程回调队列发生阻塞，导致指令在发送端排队，读取时间戳时产生严重偏移。
- **解决对策：**下周将通过 nodelet 异步规划架构进行根本性修复。

消除“起飞纠结”与“初始抖动”

- **现象：**三机基线下，发现 Drone_0 在任务起始阶段存在原地打转、响应迟滞的现象。
- **技术修复：**
 - a. **Yaw 映射修正：**排查发现 `traj_server` 的偏航角计算语义存在偏差，导致控制器将绝对角度误认为角速度。现已建立严格的 `yaw_rate` 与 `yaw_dot` 隔离机制。
 - b. **A* 窗口动态补丁：**针对高速起飞时的特征点不稳，建立了启发式的 A* 窗口自动扩展逻辑，确保在初始位姿存在微小扰动时规划器仍能给出合法路径。

尽管如此，目前10m/s仍然存在些许不稳定，drone2数值稳定性不足，导致从1.2m/s到5m/s的到达时间从31.3秒下降到10.1秒（没有达到4倍是因为无人机的SEQUENTIAL_START逻辑。另外，当前基准地图中起点终点直线距离为30米），但是5m/s到10m/s（9.5s到达），差异不足1s，drone_0打转现象仍存。目前仍需要确定是由于仿真不稳定，还是优化没有到位。



下周科研规划：从“工程闭环”转向“性能突破”

在完成 10m/s 基准探测并识别算力瓶颈后，下周计划实施以下针对性改进：

- **算法端（追求极速稳定性）：**

- a. **引入异步重规划逻辑：**重构 FSM 状态机，实现规划循环与控制循环的线程级解耦，消除 10m/s 下 CPU 瞬时负载过高导致的控制指令断流。
- b. **非均匀分辨率 GridMap：**针对高速飞行设计“前长后短”的感知滑动窗口，并对远端栅格进行启发式降采样，以压低 A* 搜索的搜索空间。

- **感知端（追求环境鲁棒性）：**

- a. **模拟传感器扰动实验：**参照 UZH 的研究，在深度图中人为注入高斯噪声与缺失值，测试 EGO-Planner 在非理想感知下的避障边界。

- **任务端（语义闭环准备）：**

- a. **语义层前瞻：**在 10m/s 基线稳固后，启动 VLM 调度实验，进行“语义目标搜索”的初步闭环和前置工作。

本周的工作完成了从“能动”到“能度量、能分析、能优化”的闭环。当前 EGO-Planner V2 的底层架构已在 1.2m/s 至 10.0 m/s 的宽速域内完成摸底，下周之后的优化将进一步提升稳定性。

注：陈老师（硕导）毕设：由于传感器已到货，需要产出现场数据，因此近2-3周可能会相对保持慢速迭代（~20%）。建议重点落在可靠性的提升上。

GAZEBO瓶颈 => 高速 Isaac ① 补丁

更具体的快速覆盖 ② => 调研为主

现在坑解决

1.未到达结束约束 2.产生原因，planner约束，分析。

2026年4月2日

本周基于 Fast-Drone-250 “多机决策深度解耦”的设计哲学，彻底重构了系统的多机部署架构与规划内核管理机制。通过清理此前积累的“技术债”，目前系统已实现从底层物理仿真到高层算法调度的全链路闭环，标志着代码决策链将不再引入主要修改，为后续实机部署与语义搜索任务打下了坚实的软件底座。

多机扩展：绝对规范，逐层封装，话题清洗

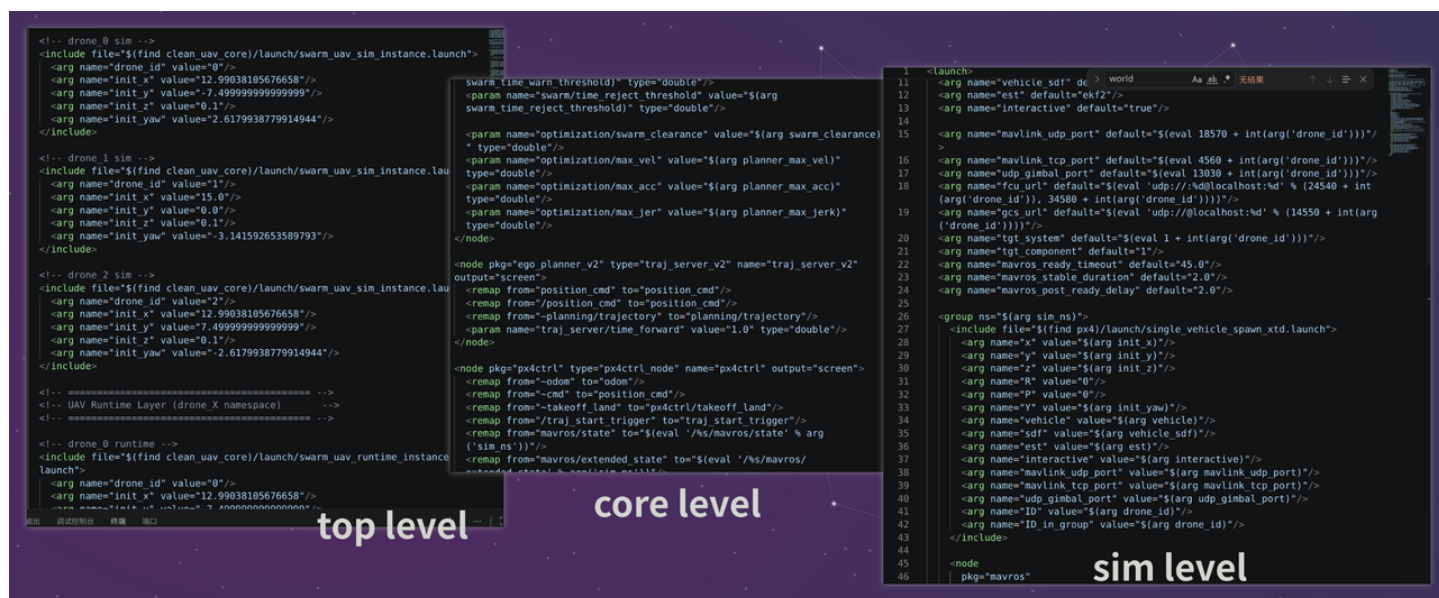
在仿真中，由于launch文件没有for循环，因此在最顶层的launch,不能避免复制粘贴。在之前的实现中，还存在下面的问题：

- 顶层文件耦合过多：由于存在大量的时序层面的依赖，且时间分配固定，状态机写死。
- 文件分工不明确：对于仿真、规划控制、感知多个层面互相混杂。

- 实现割裂：命名空间混杂（仿真的 `iris_0` 和 `drone_0` ），从而导致了大量的话题重映射，阅读杂乱。

基于此，本周进行了【洋葱式】的架构拆解。具体来说分为下面的部分：

- 顶层：由 Python 自动化脚本生成，仅负责全局场景定义（Gazebo World）与任务分发。
- 中层：核心解耦层。封装了 `ego_planner`、`px4ctrl` 等核心算法包。该层实现了“环境无关性”，代码无需修改（或只需调整IP地址）即可直接部署于 Ubuntu 20.04 + ROS Noetic 的真实机载电脑。
- 仿真层：针对 Gazebo SITL 进行了物理特性的深度适配。



关键技术改进点：

- 全链路命名空间清洗：废弃了所有混杂的命名空间，强制所有话题共享 `drone_${arg drone_id}` 前缀。通过设置 `robotNamespace` 参数，彻底消灭了冗余的 `topic_tools/relay` 节点，大幅降低了多机环境下的 CPU 消耗与通信延迟。
- 实现“就绪门（Readiness Gate）”机制：废弃了不可靠的硬编码延时，开发了基于 MAVLink 状态感知的触发脚本。系统仅在物理连接、里程计初始化及飞控握手 100% 确认后方可激活规划算法，极大地提升了分布式系统的启动确定性。

V1/V2整合：彻底分离，全部工作良好

核心设计：完全分离

基于上周确定的解耦基本原则，本周再向前走一步：

- 所有胶水层不再试图编写if-else，而是完全独立开发
- 不再通过条件编译区分包，而是直接通过更改ego-planner-v2，给所有包加上v2，进行严格的版本管理和区分。

上述关键确认保证了：不会因为任何层面的话题不对齐，或者考虑遗漏导致适配不完全。本周因此发现，gridmap读取不正确，是早先针对于v1版本的gridmap预处理的一个侵入式修改中，在良好的开发习惯中也得到解决。

集群协同优化：时空一致性与参数精调

当前的 `clean_uav_core` 的 V1 与 V2 顶层入口都已使用全局共享广播 topic，而不是每机各自的隔离 topic。基于此，V1 与 V2 都会在规划成功后持续发布自身轨迹，优化器也都会读取 `swarm_traj` 缓冲区计算 swarm cost。对于轨迹判定“正常”的行为：

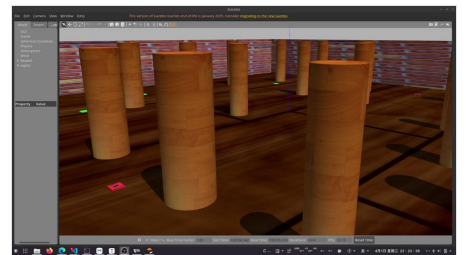
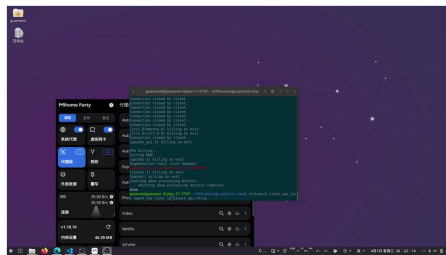
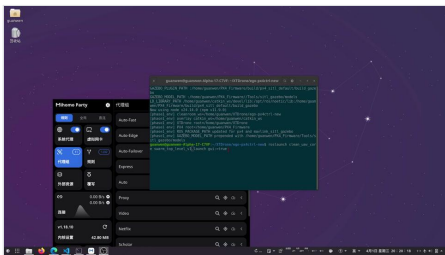
- V1 当前会在接收回调中直接丢弃时间差超过 0.25s 的轨迹，并且会把距离自己超过 $\text{planning_horizon} * 4 / 3$ （默认等于10米）的轨迹直接标记为无效。
- V2 当前虽然时间策略较宽，但仍会把“暂时离自己较远”的轨迹作废；同时 V2 默认 `swarm_clearance=0.15`（集群距离代价），对三机交汇场景过小。

可以看出，在不做任何更改的情况下，三机交汇会导致存在部分碰撞情况，而更改之后，UAV出现了明显的先后通过现象。体现了强大的优化效果（见下面三个视频）。

最后效果：V1/V2正常，3-6机交汇实验均通过

本周顺利完成了不同规模下的集群避障测试，系统表现稳健：

- 视频 1：3 机交汇（无集群避障，基准对比）。
- 视频 2：3 机交汇（开启协同优化，展现平滑分流）。
- 视频 3：6 机超大规模集群在密集障碍物环境下的鲁棒飞行。



下周建议：实机适应性开发与“语义”过渡

本周的实施标志着ego-planner+px4ctrl+vins彻底完全闭环，所有功能均开发完毕。下周主要目标是链路收尾，和继续针对于实物的适应性修改。

- 继续推进架构实物化残余事项。
 - 实现起点自动标定逻辑：使集群相互位置获取不再依赖 Launch 文件的预设坐标，改为基于实时里程计与轨迹广播的动态对齐。
 - 执行YAML分离，针对真实物理环境的推重比、震动噪声进行参数初探。
- 基准测试脚本建立。建立基于 `rosviz` 的全自动分析脚本，定量统计轨迹平滑度（Jerk 积分）与计算耗时。

- 极端场景测试：在仿真中人工注入通信丢包与传感器高延迟，测试心跳看门狗的介入时机。
- 根据实物抵达情况，视进度决定下一步部署计划。

进入week7后期和week8前期，对于仿真层面在后续的实施侧重，请李老师指导：

- 首先接入LLM => 更侧重于语义调度，场景3D图构建等（后续）
- 首先迁移到Isaac => 更侧重于定义范式，继续基建（×）
- 更高速度，统计，速度（10m/s）=> 地图，膨胀地图更新频率上升，决策能力**榨干**
- 挖掘架构潜力，深入更改架构

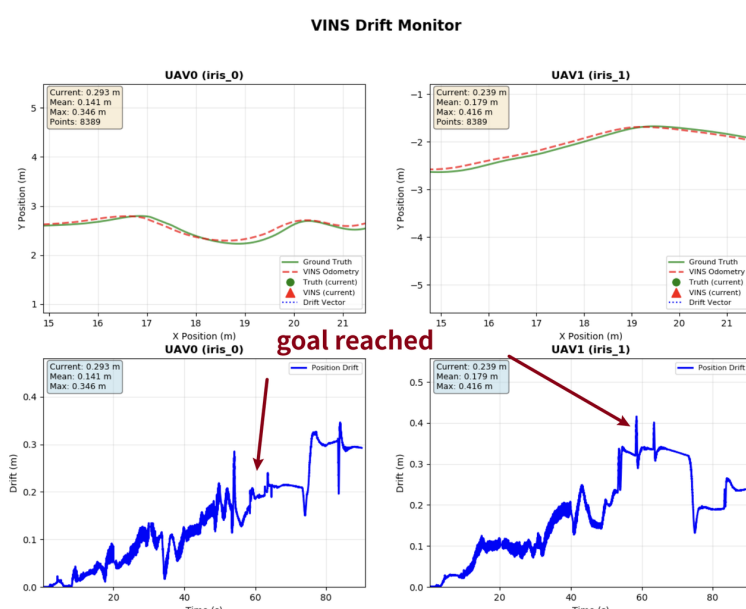
2026年3月26日

本周是week6：工作重心已由“功能实现”转向“架构固化与性能基准确立”。在经历了为期数日的底层代码重构与调试后，成功解决了由于技术债累积导致的系统不稳定性，实现了MINCO (Ego-Planner-V2) 内核的初步集成，并定量分析了定位漂移问题。

定位与规划的“回归测试”：Cleanroom最重要

上周系统出现的偶发性撞墙和 VINS 恶性漂移，在本周通过版本管理的重构得到了彻底解决。

- **Debug 洞察**：排查发现，漂移并非算法数学逻辑错误，而是由于多版本迭代中使用了软链接导致头文件引用污染，同时也存在编译污染，以及仿真环境RTF波动引起的时钟抖动（僵尸进程）。
- **成果**：在清理了所有软链接并执行全量干净编译后，目前 VINS 漂移稳定在 0.4m 以内（40m 全程），且系统起飞成功率回归至 100%。【注：下图当中60s以前为运动状态，60s后为半静止状态，5s变更一次路点】



EGO-Planner-V2 集成：大改，先打通链路

本周投入大量精力完成了规划内核从 V1 (B-spline) 向 **V2 (MINCO)** 的跨代迁移。

升级到 MINCO的关键点

- 参数表达的跃迁：V1 依赖控制点位置，本质是**空间搜索**；V2 (MINCO) 以航点和时间间隔为变量，实现了**时空联合优化**。
- 平滑度的本质差异：V1 靠增加惩罚项抑制震荡，容易陷入局部极小；V2 通过解析解保证多项式阶数连续，产生的轨迹 Jerk（加加速度）更小，对电机的压榨更平滑，是实现森林极致机动的数学前提。

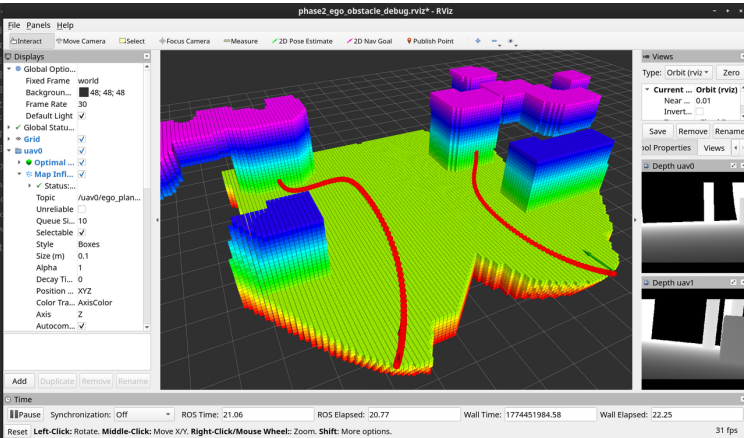
主要实施的升级点（堵点）：

- 轨迹表示存在区别（多项式/BSPLine），相应地轨迹广播机制也会引入区别
- 新增了 `planning/heartbeat`、`mandatory_stop` 等机制
- 目标的类型发生变化（"PoseStamped" for V1 (BSpline), "GoalSet" for V2 (MINCO))
- Astar的规划池显著增大，并且内部实现从固定地图变更为了ringbuffer

基于这样的大改最终决定：只复用到launch启动层，将其余的代码部分进行独立实施。

当前实施状态：

- 链路打通：已完成 `Heartbeat` 握手机制、`GoalSet` 新指令集及轨迹广播协议的重写。目前 MINCO 已能在仿真中正确生成并显示高保真轨迹，UAV也能跟随运动。
- 待攻坚点：A* 前端与 RingBuffer 的兼容性仍需微调（目前存在 A* 终点落在障碍物边缘的逻辑冲突）。下周将重点修复此“前端初值”问题。



```
guanwen@guanwen-Alpha-17-C7VF: ~/XTDrone/cleanroom_ws
4,500,50) end coord=(14.762,5.483,2.414) occ=19
[INFO] [1774451998.591672234, 35.089000000]: [GridMap] end coord: pos=(14.762,5.483,2.414) idx=(131,48,21) in inf buf=1 in buf=1 occ value=19
[INFO] [1774451998.591681371, 35.089000000]: [GridMap] inf_low=(-5.288,-11.250,-4.725) inf_up=(25.762,19.800,5.175) virt wall=1 virt ground=-0.50 virt ceil=3.00
iter=9,time(ms)=1.000, fine check collided, keep optimizing
Success=no
[FSM]Drone:0, from GEN_NEW_TRAJ to GEN_NEW_TRAJ

[35.112000000] Drone 0
[INFO] [1774451998.614859981, 35.112000000]: [Astar] Search: start_pt=(10.970,4.373,0.628) end_pt=(10.752,3.962,0.374) center=(10.861,4.168,0.501) step=0.113 PO
OL SIZE=(1000,1000,100)
[WARN] [1774451998.614887894, 35.112000000]: [Astar] end inside occ: end_idx=(50
0,499,50) end coord=(10.861,4.055,0.501) occ=17
[INFO] [1774451998.614901841, 35.112000000]: [GridMap] end coord: pos=(10.861,4.055,0.501) idx=(96,36,4) in inf buf=1 in buf=1 occ value=17
[INFO] [1774451998.614916508, 35.112000000]: [GridMap] inf_low=(-5.062,-11.588,-4.725) inf_up=(25.988,19.462,5.175) virt wall=1 virt ground=-0.50 virt ceil=3.00
[INFO] [1774451998.614990268, 35.112000000]: [Astar] Search: start_pt=(12.490,6.112,1.799) end_pt=(11.476,5.124,1.109) center=(11.983,5.618,1.454) step=0.113 PO
OL SIZE=(1000,1000,100)
[WARN] [1774451998.615004044, 35.112000000]: [Astar] end inside occ: end_idx=(49
6,497,48) end coord=(11.533,5.280,1.229) occ=63
```

集群架构设计：面向实机的分布式演进

针对多机任务，我推翻了此前“Python 脚本生成 XML”的临时方案（XTDrone分析），转而采用对齐 Fast-Drone-250 工业级标准的分布式架构。

- 重构逻辑：废弃一键启动的“大 Launch”，改为“单机 Launch 模板 + 环境变量 (Drone_ID)”模式。
- 工程意义：这意味着真机部署时，所有机载电脑（NUC/Jetson）共享同一套 Git 代码库，仅通过环境变量区分物理身份。这种“逻辑多机、物理单机”的解耦方案，极大降低了环境配置的方差。

- **集群地图**：已设计圆周交叉布置的基准地图（对标原版测试环境），用于下周验证多机 MINCO 的避障鲁棒性。

```
<launch>
  <!-- 从环境变量读取 ID, 如果没设, 默认是 0 -->
  <arg name="drone_id" default="$(env DRONE_ID)" />

  <group ns="drone_$(arg drone_id)">
    <!-- 所有节点的名称和话题都会自动带上 /drone_X 的前缀 -->
    <node pkg="ego_planner" type="ego_planner_node" name="ego_planner_node">
      <param name="drone_id" value="$(arg drone_id)" />
      <!-- 话题重映射使用相对路径 -->
      <remap from "~odom" to="/vins_estimator/odometry" />
    </node>
    <!-- 其他节点 ... -->
  </group>
</launch>
```

下周计划与实物预期管理

结合实物的进度，按需进入“软硬解耦开发阶段”。基于此，下周的建议行动如下（仿真一侧），目标是到week7可以将gazebo一侧的代码库彻底固定，便于部署到实物，或后续迁移到isaac环境中，便于开始对接语义，实现初步LLM接入。

- 多机架构完全重构：模拟分布式的架构，确保架构对齐（建议主线）
- EGO-Planner-v2链路彻底打通：修复目前显著问题，深入发掘潜在问题（支线）。

时间平衡方案（当前情况,如有实物）：

- 实物准备 (15-20%，6-8h)：主攻调试与机载环境搭建（固定投入，减小波动）。
- 算法研究 (15-20%，6-8h)：主攻 MINCO 稳定性修复与多机调度接口。

2026年3月19日（20日）

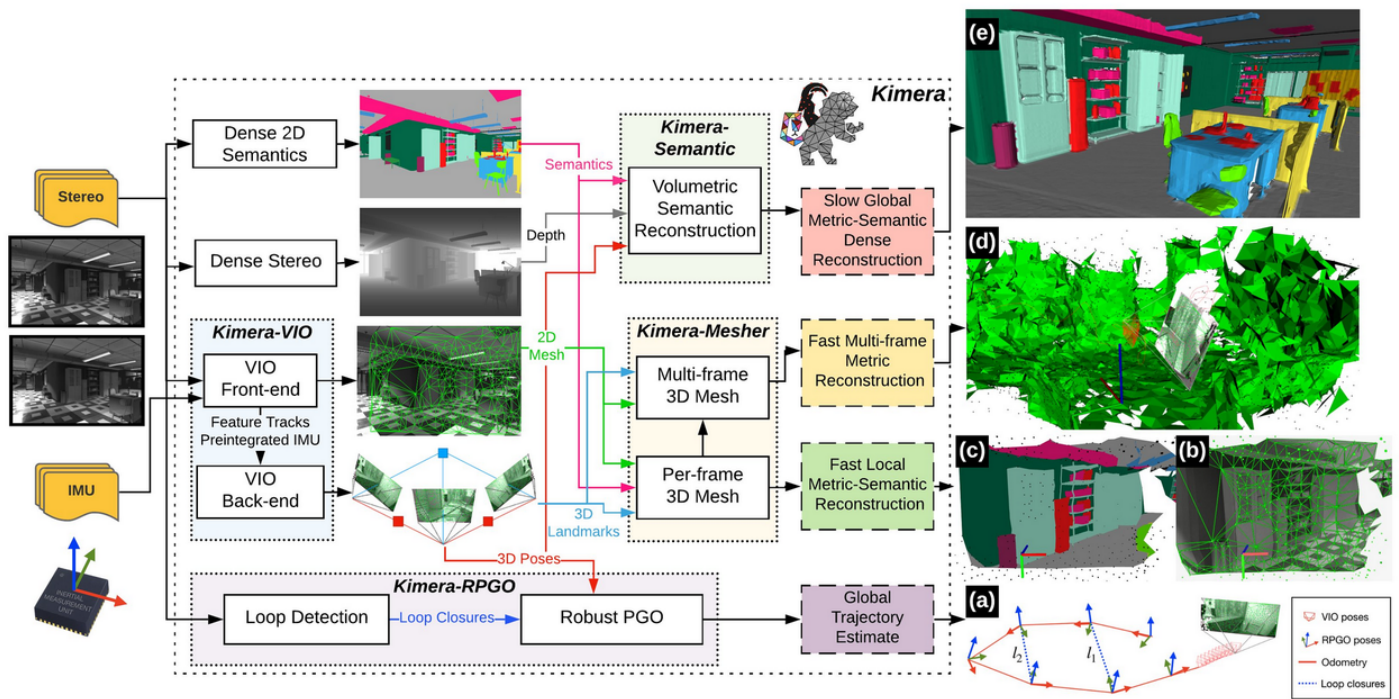
week4主要工作是：打通了VINS的链路，完成了密集障碍物的仿真实战飞行（动态航点）。

相比于上周，本周的实施工作量降低，但是调试工作量显著上升，系统的可靠性也大幅提升，积累大量经验。本次也标志着实现正式进入“深水区”。

理论：语义和3D Scene Graph初探

- 大多数对于环境的表征都必须抽象为“语义图”，几乎是核心表征模式
- 3D Scene Graph是一个非常具有挑战性的建设模型，往往与SLAM等尖端技术结合，更重视三维建模，而不是机器学习（另一个典型的可能路径是VLN）
- 最典型的文章是3D Dynamic Scene Graphs: Actionable Spatial Perception with Places, Objects, and Humans（RSS 2020）

- 代码库: <https://github.com/MIT-SPARK/Kimera> (MIT SPARK LAB)



工程实现1: VINS整合

本部分仅从实现思路来看难度不大，即：将VINS里程计输出映射到ego-planner。

主要坑点

- 必须接受 `vins_estimator/imu_propagate` (即：根据IMU短时推演的里程计，~100Hz) 作为odom，而不是 `vins_estimator/odom` (即：完全跟随相机帧的里程计回环数据，~15Hz)，否则采样率不足支持ego
- VINS进程TF坐标转换关系必须添加唯一前缀
- 为了适应VINS不稳定发送频率，需要增大里程计丢失超时，并且加入mavros里程计fallback机制，否则规划过程会被打断
- 打了补丁: `Depth Lost` 从“地图越界误触发”改成“真实深度/位姿断流才触发

工程实现2: 障碍物变换场景和动态航点

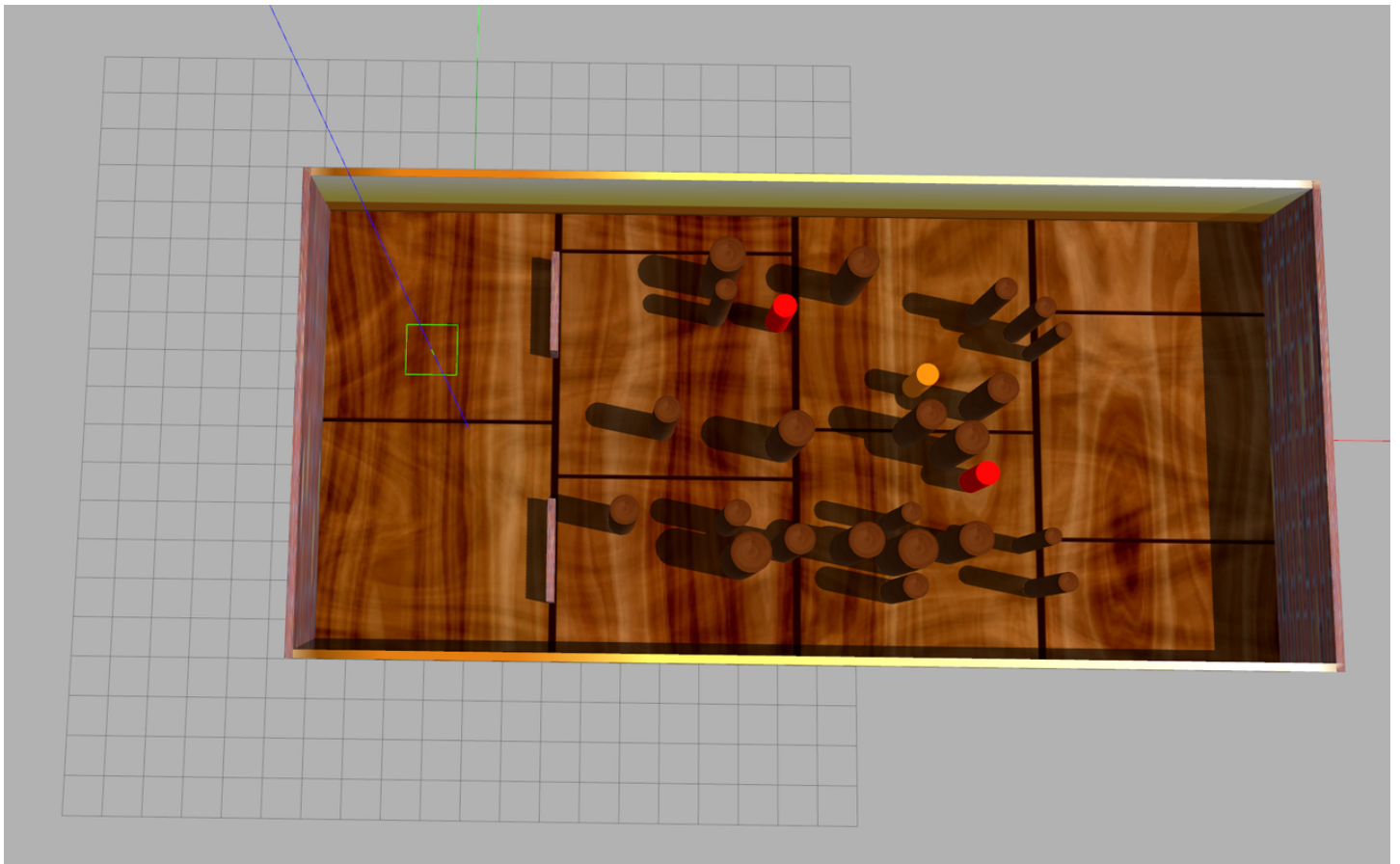
生成world文件和地图

生成了一个“柱子林”，场地大小为20m x 40m，UAV需要从x=0穿越到x=36

动态航点: x=36,y=正负2到4之间摆动(5秒发布一次,周期30秒) => 绝对不能高频发送,否则很可能反复重规划导致稳定性爆炸

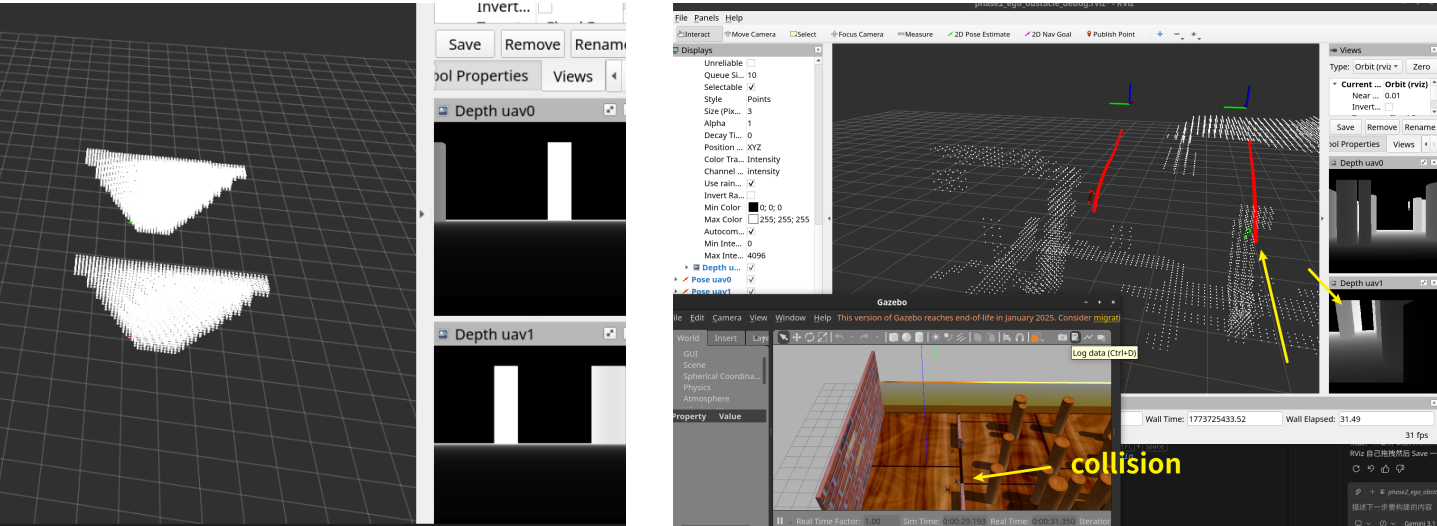
保证了纹理,确保VINS获得足够多的特征点信息。

注意: 动态障碍物由于gazebo的BUG,深度相机和实际物理碰撞存在不一致问题,因此尚未启用



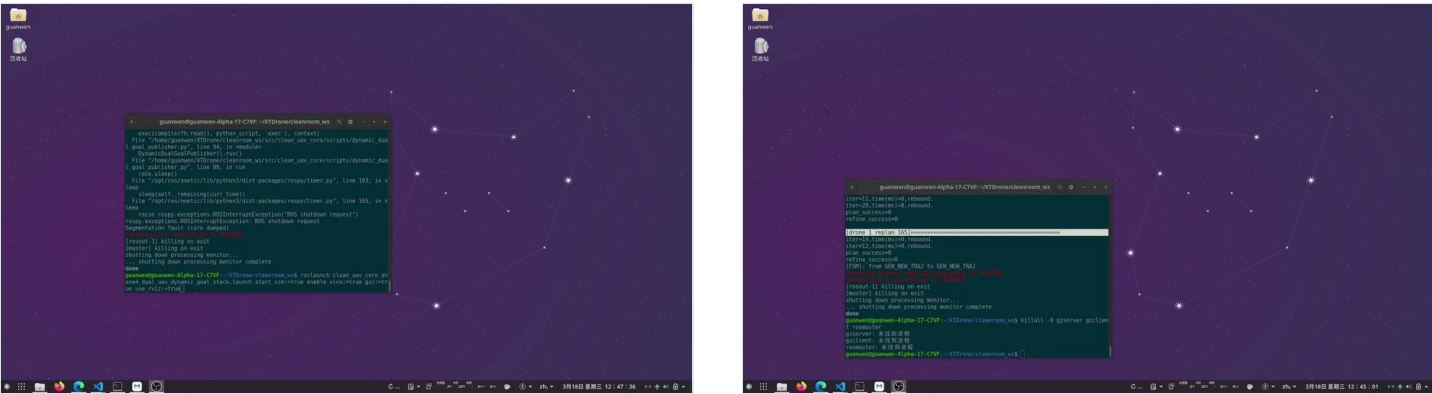
上周尚未考虑的坑点

- 1.点云需要注意：从相机坐标系迁移到世界坐标系（否则会出现诡异点云形状，彻底失效）
- 2.一定要消除掉附近0.4m的点云，否则会把自已的桨叶当成障碍物（但是这可能导致障碍物碰撞后深度相机误以为无障碍导致出错，需要定义状态机或者权衡数值）
- 3.必须做好坐标系初始化，默认vins和规划都是基于原点开始的，若不区分（初始坐标没对应绝对的世界坐标系坐标）会导致两个无人机一开始就重合到0点。
- 4.地图扩大之后，必须更改ego参数，其内部也有一个地图边界（X_max等），不修改必越界，此时目标不生效
- 5.动态goal必须关闭掉 `SEQUENTIAL_START` 逻辑（需人工接管）



视频结果： workflow 打通，但稳定性还可增强

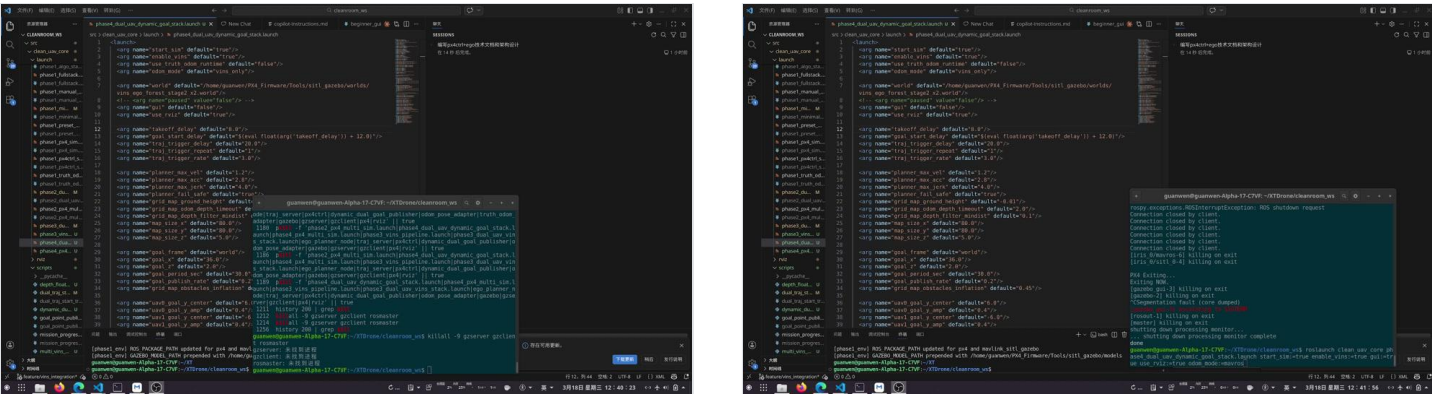
本周视频如下：视频1表明已经完成了全链路任务，但是最后VINS漂移，未能在目标点稳住。视频2使用px4里程计（mavros）替代了vins，顺利完成全程任务（也是现实当中一个高度可行的方式，但是无回环，存在稳态误差）。



本周所发现的问题，表明在广泛任务下的可靠性还需要进一步增强：

- VINS偶发漂移，导致目标点“落入”障碍物，此时无人机会撞墙（主要是静态飞行情况下）。但相比于最初调试，有显著好转，不再恶性发散
 - 目前暂时使用PX4飞控的里程计，稳定性强于VINS，但是没有长时保障
- 深度相机在无人机进入阴影的时候，或者面对一堵墙，明显稳定性下降（膨胀栅格不稳定），可能需要更长视野的重规划策略（当前最大问题）
- 基于BSPLine的策略观察视野不足，容易贪心陷入不安全策略（这也和障碍物搜索距离有关系），没有硬躲避能力

其中典型失效场景见视频3/4，其中视频3是贪心失效（源于深度图的变化），视频4是深度图失效（变黑）。



调试心得

- 本周深刻意识到，问题之间存在显著的耦合关系，不能只看表象，必须逐个先行排除，因为一个BUG便可能占用不少时间。

- 一定要拿到可视化结果和明确错误区分点，AI工具分析能力还是太弱（这方面纯AI问答远远比编码助手做得好）

week5/6主要计划

week5主要的目标是继续调试，稳定性增强，确保推进到实物之前尽可能积累经验，将大多数问题扼杀，并使用量化方法建立测试性能基准，为后续参数和方案抉择打下基础。下面是任务清单：

- 为了源码参考方便，我们使用了EGO-Planner-Swarm（基于BSPLine），下周将迁移到MINCO（EGO-Planner-v2），主要需要调整轨迹沟通方式、PX4Ctrl桥接层。
 - 同时系统对比二者之区别，建立起基准测试（与实物 workflow 对齐），量化衡量【谁更好】。如有余力，扩展到 ≥ 3 架无人机测试
- 系统分析VINS漂移问题，如不行可再行调研（或者更改光源属性，目前来看竹林和Gazebo光照条件区别显著）
- 进行进一步的调试，在密集障碍物的鲁棒性提升（消除掉死胡同、贪心等核心问题）
- 【支线】继续调研3D Scene Graph，尤其是Github的可能实现

如果week5的执行相对顺利，建议week6更多聚焦于更多恶劣场景探究系统边界，预演可能问题（例如VINS突然失效，噪声加码，算力与实时性保证），并且对于系统进行重构（例如yaml分离），确保架构实物ready。week 6根据未来两周的传感器性能对比，辅助确认采购的性能指标，从而确定大致清单。

2026年3月12日

本周是week3，主要工作：基于上周的两个开源仓库的指引，**使用cleanroom原则，从零搭建复现了gazebo -> ego -> px4ctrl -> px4的全流程**。本周为之后的技术栈对齐打下坚实基础。

理论阅读（文本）

本周主要进一步阅读并且确认基础数学原理。下面是部分内容。（公式无法显示，采用本地markdown截图）

在这篇论文发表之前，学术界和工业界普遍使用欧拉角（即 Roll, Pitch, Yaw）来表示和控制姿态。当飞机的 Pitch 角达到 90 度（机头垂直朝上）时，数学公式的雅可比矩阵会变成奇异矩阵（分母出现 $\cos(90\text{deg})=0$ ）。这意味着传统的 PID/LQR 控制器在做大姿态机动（如翻滚、极限避障）时，数学计算会直接崩溃，致飞机在空中解体（失控）。

Lee 的伟大贡献：抛弃欧拉角，直接在流形 SE(3) 上做控制。

- SE(3) 指的是特殊欧几里得群（包含三维平移 $\mathbb{R}^3$ 和三维旋转 $SO(3)$ ）。Lee 提出，不要去算 Roll/Pitch/Yaw 的误差，而是直接计算旋转矩阵 $\mathbf{R}$ 的误差！他定义了流形上的姿态误差公式： $\mathbf{e}_R = \frac{1}{2}(\mathbf{R}_d^T \mathbf{R} - \mathbf{R}^T \mathbf{R}_d)^\vee$ 。这个误差公式最终直接映射到PWM层级（PX4完成）
- 误差公式从推导

实际上PX4ctrl做的事，就是计算 $\mathbf{R}_d$ 。

安全走廊：从多面体到球体

- 为了适应L-BFGS（无约束优化器），圆柱内可通过同胚映射转化为无约束量
- 多面体也可以转换为球体。
 - 凸多面体的内点满足 $\mathbf{q} = \mathbf{v}_0 + \hat{\mathbf{V}}\mathbf{w}$ ，其中 $\mathbf{w} \geq 0$ 且 $\|\mathbf{w}\|_1 \leq 1$ （所有元素之和小于等于1）。
 - 算法引入了一个新的代理变量 ξ ，令 $\mathbf{w} = [\xi]^2$ （逐元素平方），此时约束就变成了 $\sum \xi_i^2 \leq 1$ ，这就变成了unit ball。
 - 最终借助球极投影的逆映射去除掉约束

净室开发进度

仿真器选择

仿真器继续使用Gazebo，但是不再依赖于XTDrone，而是以 `make px4_sitl gazebo` 为起点，全新开发。核心是保障系统解耦，可维护。

- Isaac配置要求相对较高，难以支持多机仿真（RTX4060）。
- AirSim老旧，Flightmare由于解耦了渲染和控制，并且PX4需要外挂。
- Gazebo最接近于实际部署的工作流。

相关原理概括

相比于XTDrone的实现，简单来说区别如下：

- XTDrone默认通过其自带的通信总线中继话题 <= 上周不稳定的根因在此！
- 我们所作的：1.在仿真器拉起和VINS启动层面，借用XTDrone的launch，但是大幅更改话题remap名称，保证对接正确 2.将ego的话题转发至px4ctrl，随后px4ctrl转换为PX4指令

重要开发内容概括

本次开发设置了一个独立的节点，作为一个胶水层：



因此核心就是，顺次启动节点，重映射话题。

核心顺序如下：

1. 解析全局参数
2. 为uav0和uav1分别启动独立命名空间（`/uav0/`、`/uav1/`）下的单机栈：
 - 接收真值里程计（`truth_odom`）作为状态输入。
 - EGO-Planner根据目标点生成轨迹（`planning`）。
 - 任务监控（`mission_monitor`）检查条件并管理任务状态。
 - PX4控制器（`px4ctrl`）接收位置指令（`position_cmd`）控制无人机。
3. 启动双机轨迹触发器（`dual_traj_start_trigger.py`）：
 - 延迟 `traj_trigger_delay` 秒后，以 `traj_trigger_rate` 频率向 `/uav0/traj_start_trigger` 和 `/uav1/traj_start_trigger` 发送触发信号。
 - 这一步最核心的目的是延迟向ego规划器发布里程计信息，从而“拖延”其开始规划的时间，本质上目的是为了顺次启动节点。
4. 各无人机任务监控收到触发信号后，开始执行轨迹任务。

数据流：

truth_odom (真值) → EGO (规划) → planning → mission_monitor (任务管理) → position_cmd → px4ctrl (控制) → MAVROS → PX4飞控

下面是时序表：

t=0s Gazebo 启动，加载 iris 无人机模型，PX4 SITL 启动，连接 MAVROS

t=3s truth_odom_adapter 开始桥接里程计，ego_planner_node 初始化（等待里程计）

t=5s px4_param_bootstrap 设置 PX4 参数（禁用RC丢失保护，设置 COM_RCL_EXCEPT 至4，以及 COM_RC_IN_MODE 至4从而不依赖遥控器）

t=10s takeoff_land_trigger 发送起飞指令

t=11s px4ctrl 请求 Offboard 模式，电机怠速 3s (实际上需要5-6s)

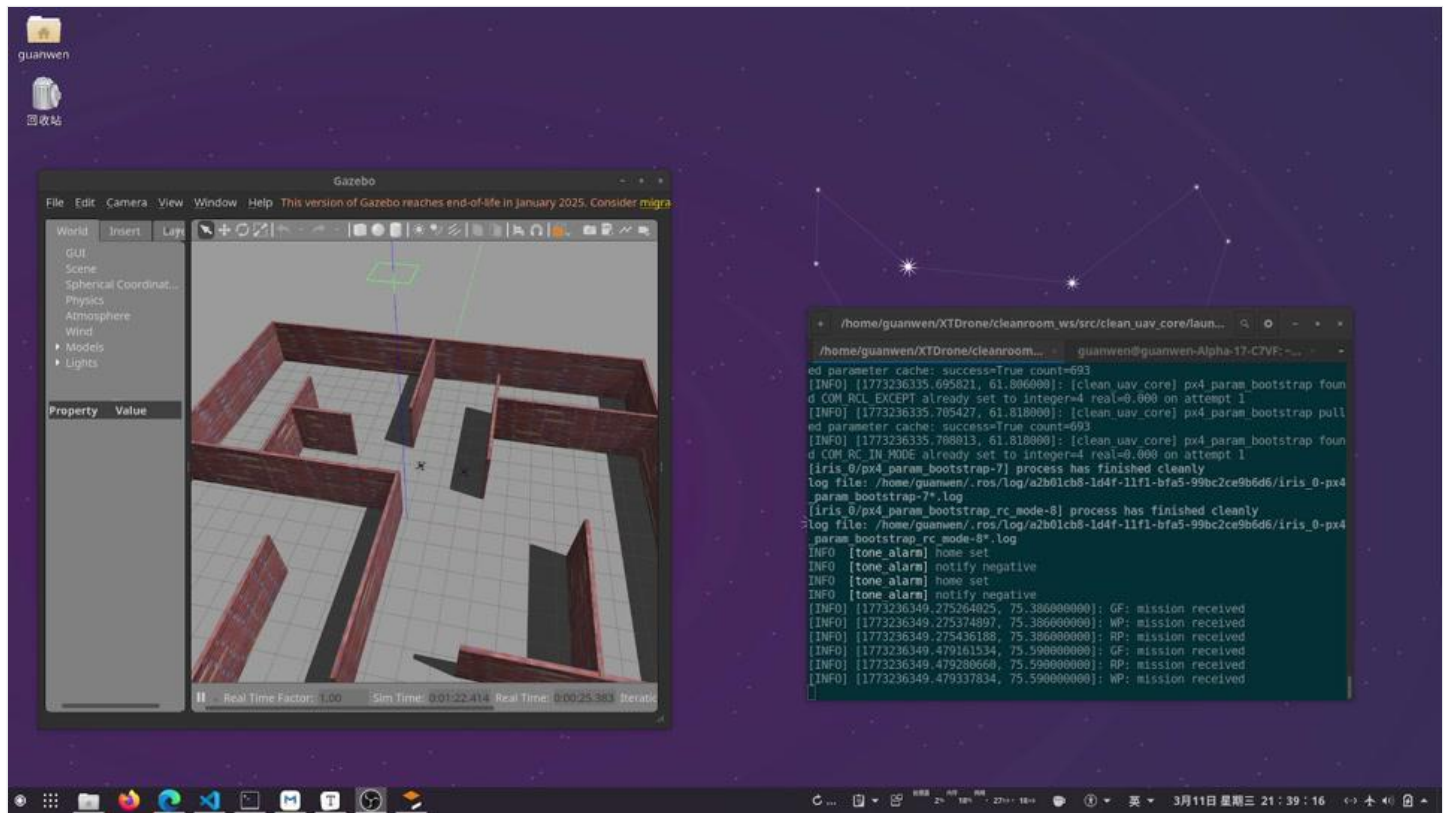
t=16s 无人机爬升至 1.0m 目标高度

t=17s traj_start_trigger 触发规划器

t=17s ego_planner 开始规划到目标点，无人机开始飞行到目标

当前结果：多机流程彻底打通

参见视频。目前已经达成：1.双机起飞和协调；2.EGO-SWARM顺利完成双机规划（两航点）。
相比于常规XTDrone需要开启7个launch，本次实现大幅整理了结构，只需要分别启动gazebo,以及规划层两个launch。



实现上与实际机器的差异

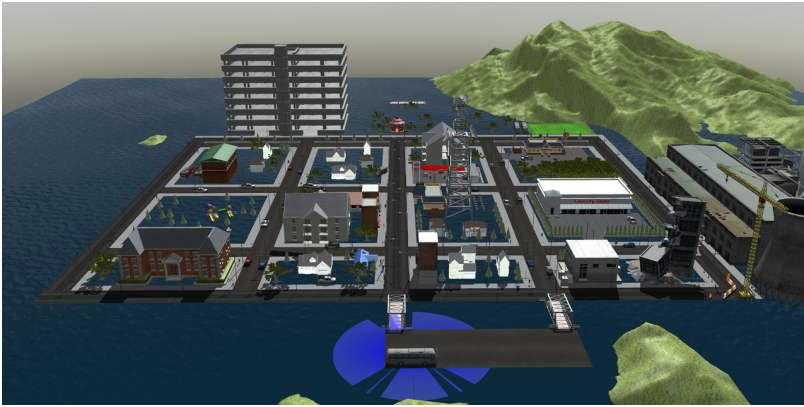
除了前述的NO_RC之外，典型的差异概括如下：

- 由于仿真机能限制，暂时降低了IMU的判定阈值（120Hz -> 80Hz）
- 如果OFFBOARD丢失之后，则短暂切换 POSCTL 再回OFFBOARD，避免offboard断开。
- 目前暂时只适配了iris，其他机型话题名称需要对齐
- 仿真中适当提高了HOVER_PERCENTAGE从而加快起飞速度（尽管这只是优化初值），实际当中应当从低数值往上逐渐测试

下周计划

下周主要是完善本周尚未完成的流程。

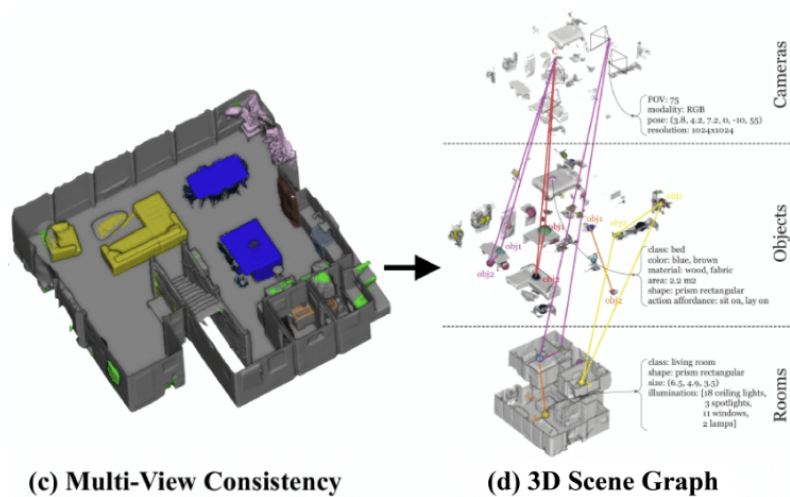
- 动态目标的指定：
 - 无人机可以跟随动态运行的轨迹，或者航点。
 - 相应开发可视化封装层，作为性能监控以及LLM的输入。
- 本任务主要为后续LLM整合定基础，因为LLM本质上就是结合定位航点和全局的感知信息，作出判断。
- VINS的引入。尽量以定量的方式评估里程计的使用，为现实部署积累经验。
 - 与之伴随，迁移到outdoor场景。



1.障碍物密集 2.动态

结合**3D Scene Graph**进行调研。

- 这一任务目前是支线。这是后续集群共识形成可行的方向。【MIT SPARK Lab (Luca Carlone 团队)】(所谓“语义”)



尽量在week 5或week 6左右开始进行实物部署，届时和老师讨论。

AI生产力分享

- github copilot的多种用途: `copilot-api` => 将github copilot的价值压榨到极致

2026年3月5日

本周是week2，主要工作：1.继续阅读EGO源代码，主要集中在状态机和轨迹优化，结合此进行初步复现 2.开始复现XTDrone和Fast-Drone-250两个新的开源项目

MINCO继续阅读：

状态机

1. **INIT**：刚开机，等待里程计 Odom 唤醒
2. **WAIT_TARGET**：等待目标点 Goal
3. **SEQUENTIAL_START**：集群专属起飞逻辑。（如果有多架飞机，需要排队启动）。
4. **GEN_NEW_TRAJ**：接到新目标，第一次生成前往目标的完整轨迹。
5. **EXEC_TRAJ**：核心状态！沿着轨迹飞
6. **REPLAN_TRAJ**：重新规划一段新轨迹。
7. **EMERGENCY_STOP**：紧急悬停制动。

其中EXEC会做出如下判断（100赫兹）：

1. 终点在障碍物的判断 (`mondifyInCollisionFinalGoal()`)：如果环境更新了，原来的终点被占据，它会沿着旧轨迹往前“倒退”寻找安全点，把安全点作为新终点
2. 如果当前时间超过了轨迹总时间，说明飞到了。清除目标，切换回 `WAIT_TARGET`
3. 滚动时域规划
 - 超出 (`replan_thresh_`)，主动触发 `REPLAN_TRAJ`，再往后算一段新的轨迹

`REPLAN_TRAJ`利用 `t_cur` 在当前正在执行的轨迹上进行采样，以此作为下一段新轨迹的起点！这样拼接出来的两段轨迹，在数学上是完美连续的。**注意：不使用odom信息，因为噪声很大，体现在acc/jerk上尤其明显**

安全措施：

1. 扫描当前轨迹
2. 静态查碰撞
3. 动态查集群
4. 一旦发现碰撞：
 - 不进入`REPLAN_TRAJ`，直接调用 `planFromLocalTraj()` 函数绕路
 - 如果**算不出**安全轨迹：
 - 如果撞墙时间小于阈值，立刻将状态机打入 `EMERGENCY_STOP`

- 如果还有缓冲时间，将状态机置为 `REPLAN_TRAJ`，让主循环接着想办法

EMERGENCY_STOP：直接生成一段只有两段、长度极短的假轨迹，其终点位置就是当前位置，速度和加速度强制写死为0。如果可行，会切换回GEN_NEW_TRAJ

优化与初步探究

$$J_total = J_smoothness + J_obstacle + J_swarm + J_feasibility + J_variance + J_time$$

目标项	代码变量	权重参数	默认值	作用
平滑性代价	内置于MinAccOpt	无（基准=1）	-	最小化snap平方积分（s=3）
障碍物避障	wei_obs_	weight_obstacle	10000	硬约束：避开静态障碍物
软性障碍物避障	wei_obs_soft_	weight_obstacle_soft	5000	软约束：远离障碍物
集群避障	wei_swarm_	weight_swarm	10000	硬约束：避开其他无人机
动力学可行性	wei_feas_	weight_feasibility	10000	硬约束：速度/加速度/加加速度限制
方差正则化	wei_sqrvar_	weight_sqrvariance	10000	软约束：轨迹段长度均匀
时间最优	wei_time_	weight_time	10	软约束：最小化总时间

上周已经成功运行复现，本周基于这一规划进行重新探索。

主要进行实验：

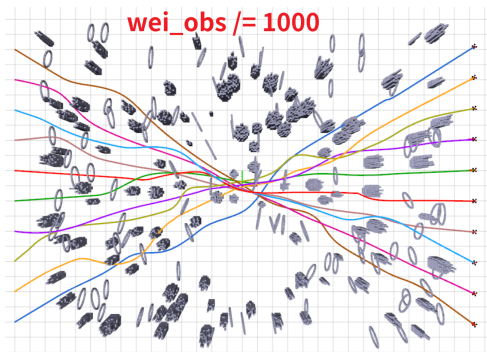
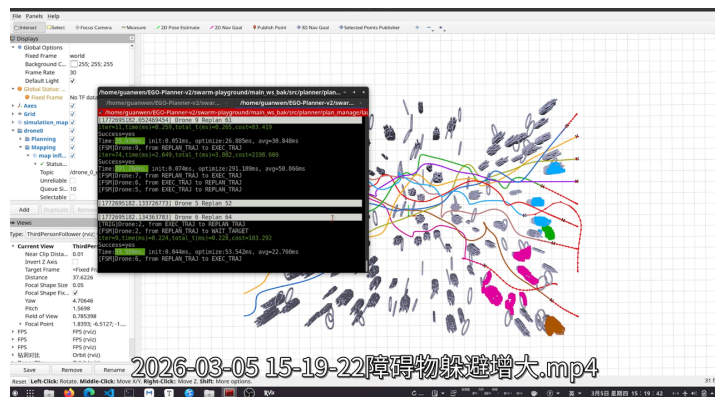
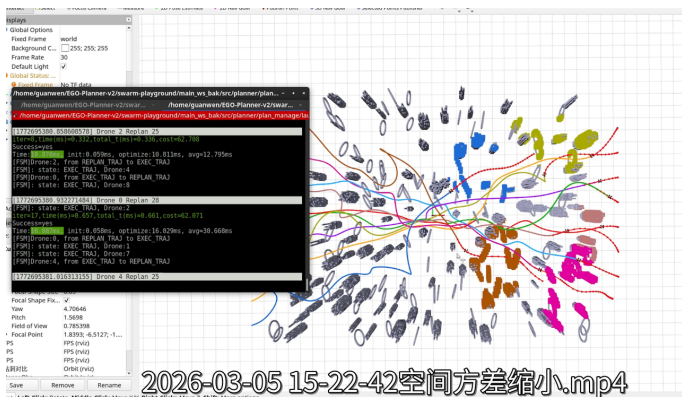
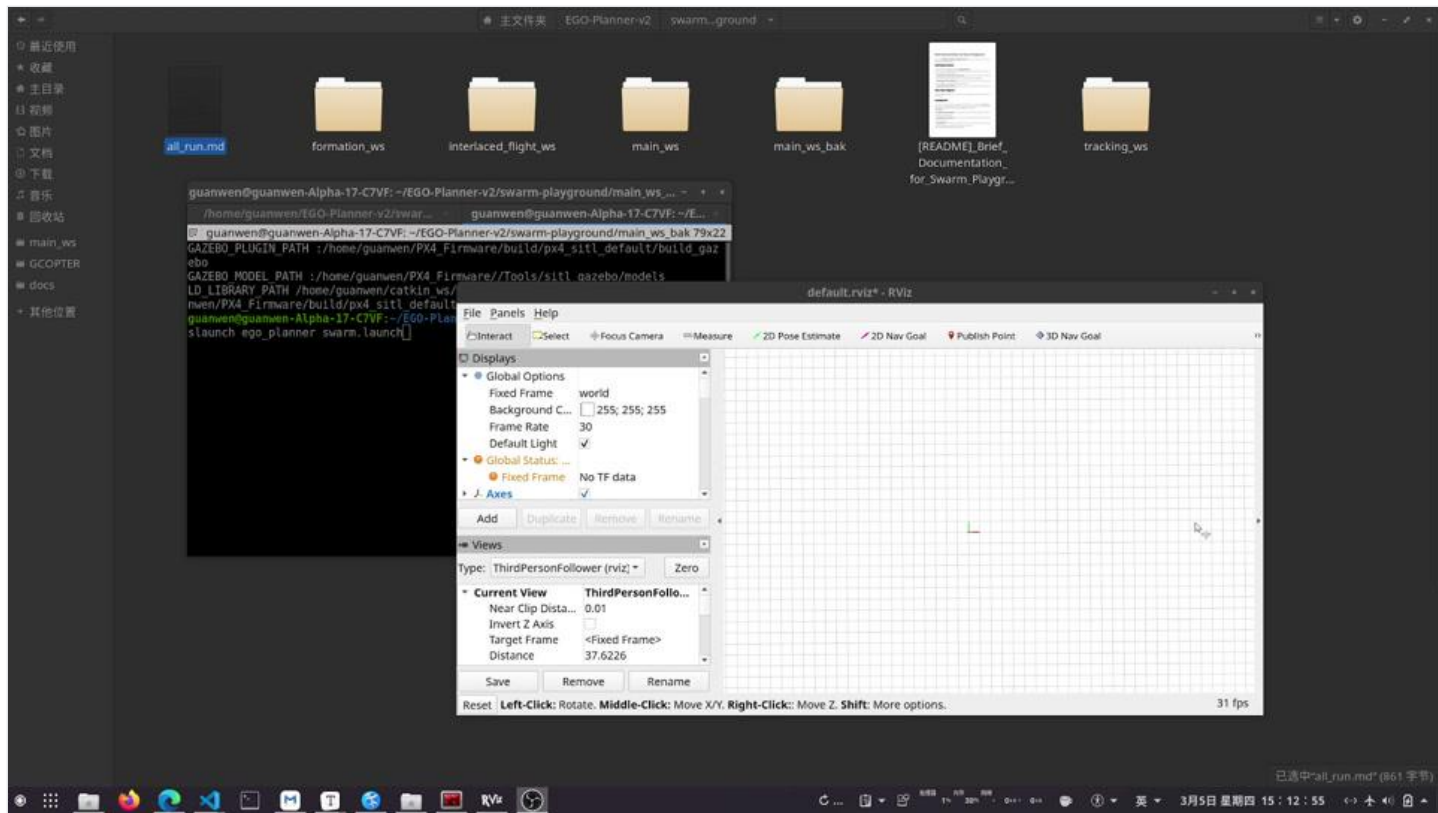
`weight_sqrvariance`：路点空间均匀，本项对于规划影响最大

`obstacle_clearance`：障碍物距离

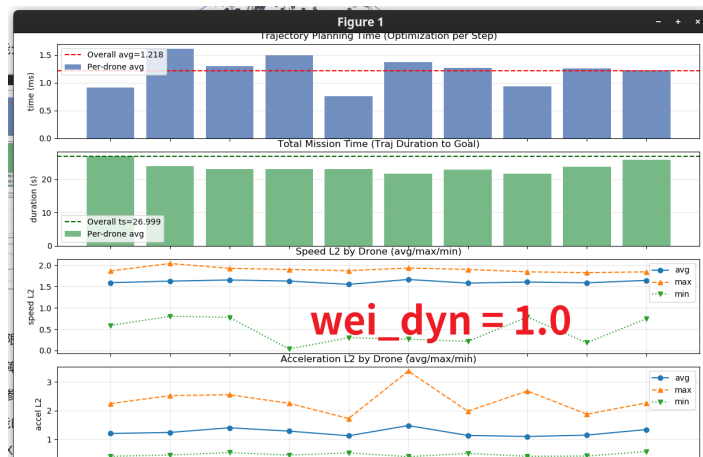
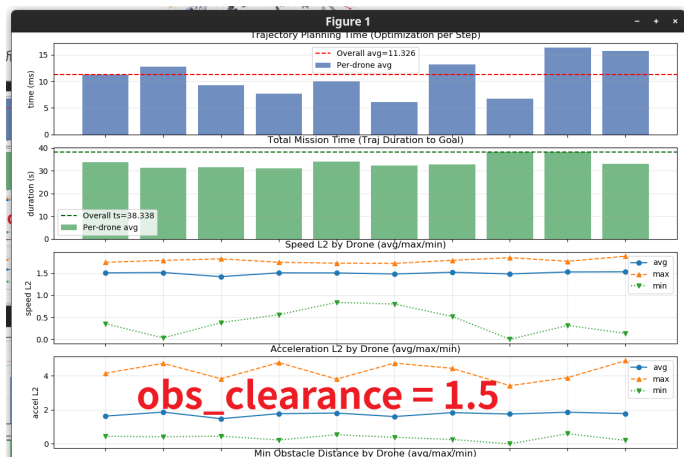
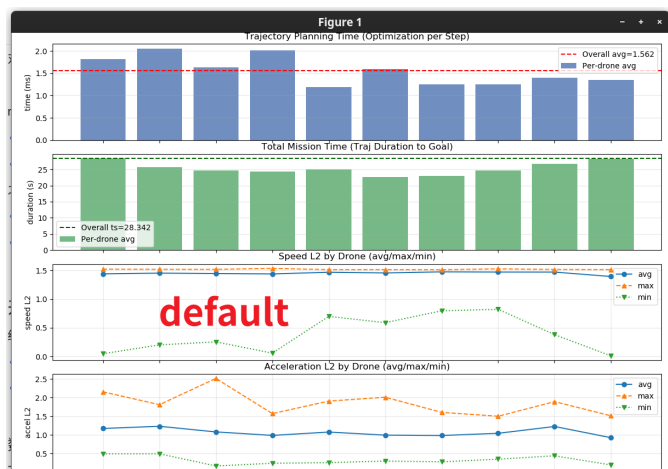
`weight_time`：耗时约束

`weight_feasibility`：动力学可行性

视频是默认情况，剩下的图片展示，视频在本地



对于性能分析主要对其汪博士论文



rviz的局限：

- 躲避障碍物的安全裕量（包括集群互相接近）近乎“榨干”，因此真正的障碍物躲避只能基于栅格膨胀

- 影响参数调节主要的因素是实验尺度，和任务时间。但是整体上对于参数不敏感

之后可能的方向：

- 结合XTDrone等的迷宫去进一步试验可行性（死胡同），并且调节尺度
- 建立标准化的测试地图

开源代码概述：基于PX4仿真流程

结合上周调研结果和检索，暂时确认了下面两个关键的开源代码仓库：

- XTDrone：一个国内社区主导的PX4 SITL（软件在环）仿真平台。目前成功仿真
- Fast-Drone-250：高飞老师在深蓝学院开设课程的配套代码，包含了完整的机器搭建流程，基于ego-swarm。主要缺点是没有SITL。本仓库不带仿真，后续考虑在XTDrone下运行起来。

数据流动：从VINS，到EGO，再到MAVROS

下面以Fast-Drone-250详细讲解这个流程是怎么进行的：

- 首先，使用VINS-Fusion这个典型的双目+IMU里程计获得里程计信息。

- **摘录自XTDrone文档：** VINS-Fusion的双目+IMU可以在静止条件下完成初始化，因此不用额外考虑。但单目+IMU以及ORB_SLAM3的单目/双目+IMU均需要在运动中初始化，在实物系统中，我们可以用手晃一晃飞机实现初始化，而仿真中必须起飞才能让无人机运动，但PX4的offboard模式如果没有定位数据，又无法起飞，这就造成了矛盾。因此仿真里只能用Gazebo真值给PX4定位，然后在飞行过程中初始化VIO，VIO的输出只用于测试精度，并不实际给无人机定位用。
- 从双目信息转换到栅格地图：直接将话题重新映射到 `grid_map.cpp` 当中（此部分内容直接包含在EGO源码中）。简要原理：外参适配、坐标转换，固定20Hz调用核心处理泵（投影、概率打分、膨胀障碍物）
- EGO输出规划结果，位于 `src/utls/quadrotor_msgs/msg/PositionCommand.msg`

```
Header header
geometry_msgs/Point position
geometry_msgs/Vector3 velocity
geometry_msgs/Vector3 acceleration
geometry_msgs/Vector3 jerk
float64 yaw
float64 yaw_dot
float64[3] kx
float64[3] kv
# 其余省略
```

- PX4Ctrl包接受规划结果和VINS里程计。
 - 内部包含状态机

```
状态：
- `MANUAL_CTRL`
- `AUTO_HOVER`
- `CMD_CTRL`
- `AUTO_TAKEOFF`
- `AUTO_LAND`

典型转换：
1. `MANUAL_CTRL` -> `AUTO_HOVER`
- 要求：有 odom、无提前 cmd、速度合理
- 动作：重置推力映射、锁定悬停点、切 OFFBOARD
2. `AUTO_HOVER` -> `CMD_CTRL`
- 要求：RC 进入 command 模式且接收到新 `cmd`
3. `MANUAL_CTRL` -> `AUTO_TAKEOFF`
- 要求：takeoff 命令触发、静止、已落地、RC 安全位
- 动作：切 OFFBOARD，必要时自动 ARM
4. `AUTO_*` -> `MANUAL_CTRL`
- 条件：失去 hover 模式、关键消息超时等
- 动作：退出 OFFBOARD
```

- 按照公式计算期望推力：

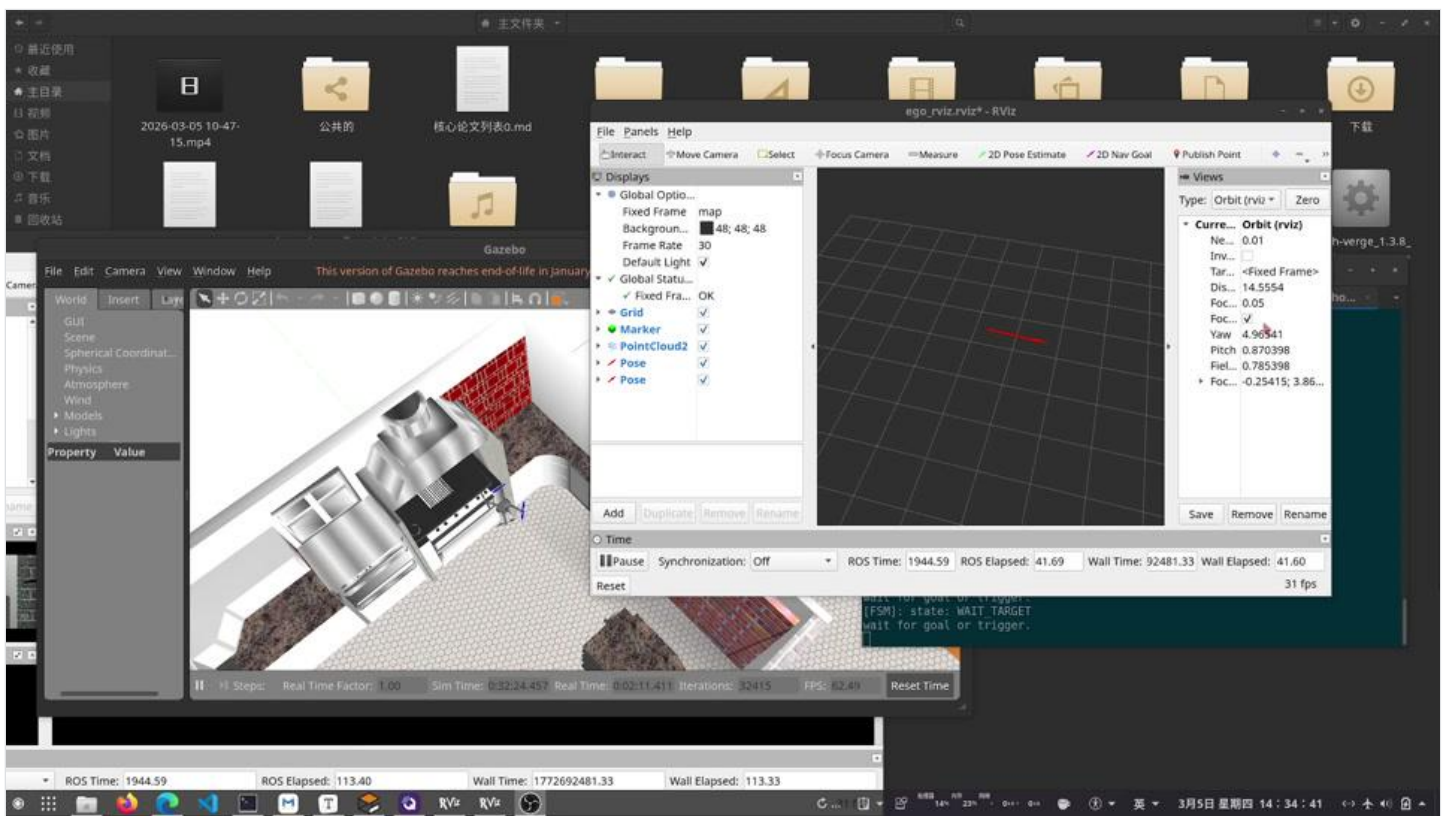
$$\mathbf{a}_{des} = \mathbf{a}_{ff} + K_v(\mathbf{v}_{ref} - \mathbf{v}) + K_p(\mathbf{p}_{ref} - \mathbf{p}) + [0, 0, g]^T$$

，简言之就是加速度跟踪，并且修正位置/速度误差（注意：*jerk*没有输入到PX4中，作为规划量存在），随后按照微分平坦逆运算获得推力与旋转矩阵（z轴->辅助向量->y轴->x轴）**这考虑了PX4自带的姿态环控制**

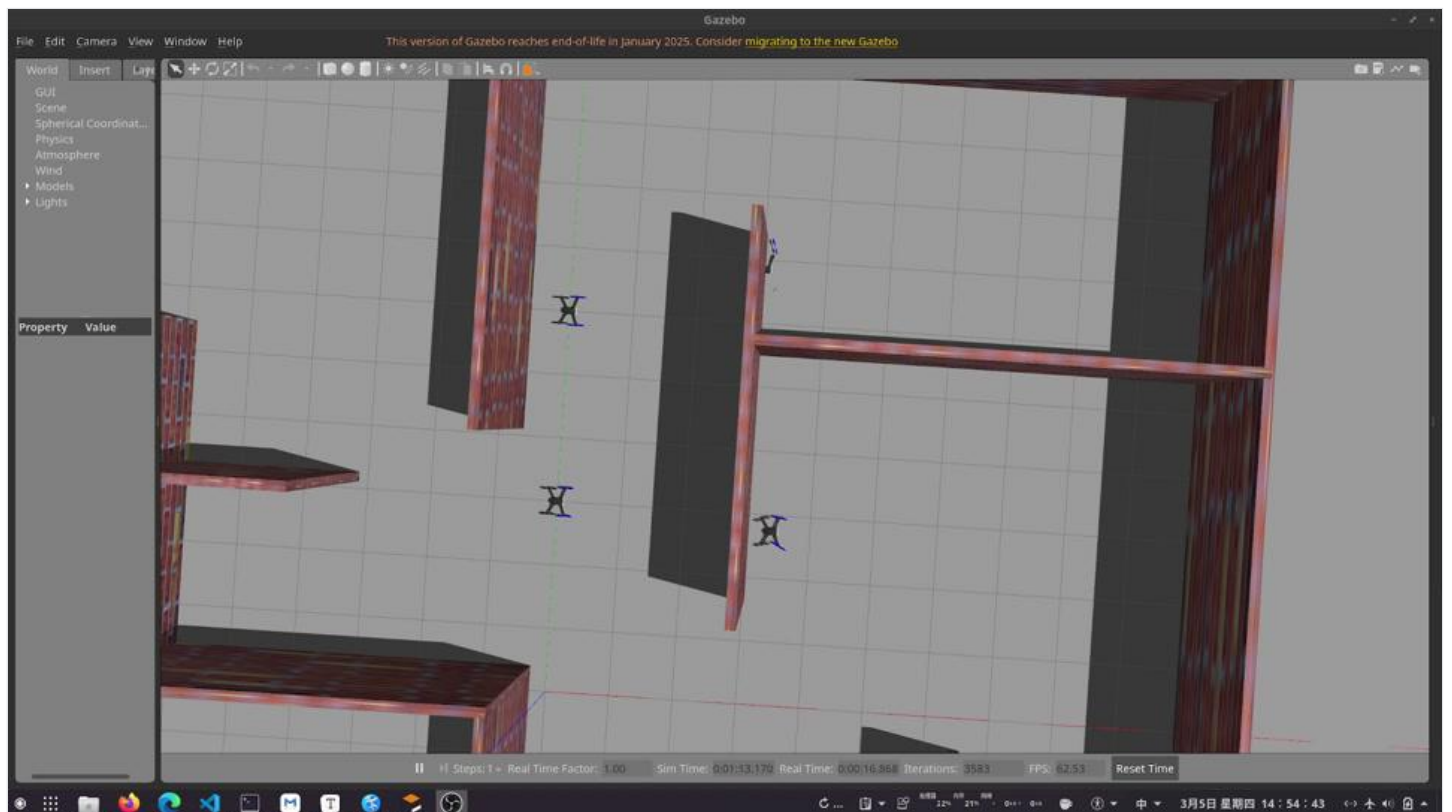
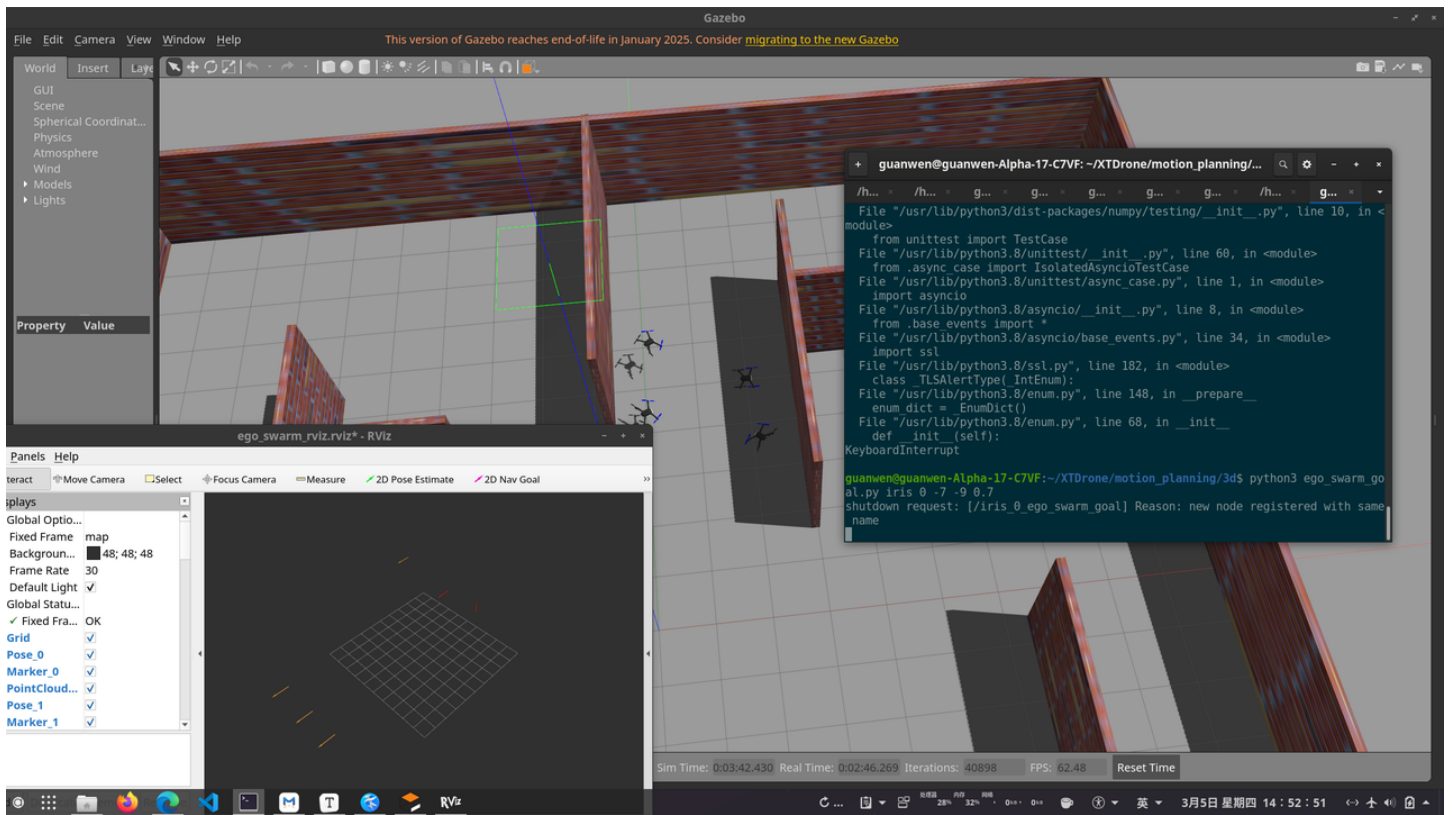
XTDrone整体上大同小异，但是由于通用性考虑，例如从ego规划结果到MAVROS的转换遵循内部量由于ROS是话题驱动的，所以可以比较方便地互相移植。目前计划是先使用XTDrone（或者是Airsim等），后续迁移到PX4Ctrl等，实现独立运作。

初步复现

按照教程操作，目前已经初步实现ego-planner的驱动。与直接运行的主要区别：1.需要打开更多节点，包括坐标系转换 2.仿真变为Gazebo，制定目标和看规划仍然使用rviz。后续的无人机失控原因见坑3（也不能排除是Gazebo仿真的BUG）



目前ego-planner-swarm的驱动尚未成功，具体体现为：起飞指令只能成功打开三架飞机，`python multi_vins_transfer.py iris 4` 指令不能驱动（表现为：没有任何输出，Ctrl+C之后退出也不会抛出KeyboardInterrupt.）。但是过程没有报错，后续可以深入排查。



踩了一些坑: => 若可以, 后续可整理为技术资料

- 1.VINS-Fusion在Ubuntu20.04安装下的兼容性问题 (解决方案: 采用社区大佬移植方案github: stevenf7/VINS-Fusion,手动也可以配, 主要是openCV和C++依赖层面的区别)
- 2.工作区调整问题 (XTDrone内置的ego_planner需要单独设立一个workspace之后编译启动)

3.VINS的部分特性和初始化问题，绝对不能让VINS处于无特征区域，否则必崩。另外由于VINS的z偏差大，一旦不使用气压计估计，起飞会崩溃

4.librealsense的安装（必须从apt源，安装2.55版本，否则识别失败，优先级高于手动make install）

5.gazebo的仿真退出不完全问题：`killall -9 gzserver gzclient rosmaster`（否则坑呢个出现process died问题）

6.QGC中设置NAV_RCL_ACT为disable（断连之后不返航），COM_RC_LOSS_T设置为35（识别为断连的时间）

7.多机启动情况下需要更改初始位置，否则生成位置卡在墙里面（视频）

还需要阅读（mark）：1.坐标系转换问题 2.目的地制定与动态开发

后续可能（据实际而定）：1.融合VIO+GPS信息（EKF加强），以及VIO选取（激光、气压计...） 2.目标选取逻辑增强（现有目标指定逻辑不清晰）

下周计划

下周主要的目标如下：

- 从数学原理开始深入攻坚，包括安全空间生成等的详细数学推导
- 结合更多仿真，成功驱动ego-planner-swarm，实战中对于PX4、QGC等无人机基本工具有更深刻理解

前瞻：week3的目标是以移植初步学习ego系列实物部署的规程。week4结合week3进展，确定XTDrone“实物流”化的更优方案，week5预计开始客制化开发。

AI生产力

- 使用gitingest+google ai studio实现代码全方位阅读和指导（1M上下文），也可用于deepseek，或者Qwen 3 Coder系列
- github教育优惠与github copilot pro（VSCode最新版）=> 300次调用权限/月，GPT 4o/5mini随使用，可用claude opus（3倍消耗量）

https://blog.csdn.net/Yu__Xin/article/details/151198414 github教育申请教程（我个人5分钟完成，3天后可领券）

2026-02-26

本周是高飞老师EGO系列复现和阅读的第一周。在接下来，将用“week 1”指代本次组会，以此类推。

基本进度：系统安装，运行，开始深入剖析

全新安装了Ubuntu 20.04 + ROS 1 Noetic实机系统，安装NVIDIA显卡驱动。

根据四篇文章的演进关系：

Fast Planner -> EGO-Planner -> EGO-Swarm -> (GCOPTER) -> EGO-Planner-v2

EGO-Planner-v2是其中集大成之作（对应SciRob），包含了集群优化、MINCO等诸多核心创新点，作为接下来代码阅读基础。

核心源码解析

从代码结构开始：EGO-Planner-v2结构上具备了系列工作的全部核心内容，下图是基本结构示意图：

planner/

- |—— drone_detect/ # 无人机检测模块（感知独立模块，一脉相承）
- |—— path_searching/ # 路径搜索（A*算法）
- |—— plan_env/ # 环境表示（GridMap）
- |—— plan_manage/ # 有限状态机（FSM）
 - | |—— src/
 - | | |—— ego_replan_fsm.cpp # 状态机管理
 - | | |—— planner_manager.cpp #
- |—— swarm_bridge/ # 多机通信桥接（压缩MINCO表示）
- |—— traj_opt/ # 轨迹优化
 - | |—— include/optimizer/
 - | | |—— gcopter.hpp # MINCO轨迹表示核心（s=2）
 - | | |—— poly_traj_optimizer # MINCO轨迹表示核心（s=3，实际使用的版本）
 - | |—— src/
 - | | |—— poly_traj_optimizer.cpp # MINCO轨迹优化核心
- |—— traj_utils/ # 轨迹工具
 - |—— include/traj_utils/
 - | |—— plan_container.hpp
 - | |—— planning_visualization.h
- |—— msg/ # ROS消息定义
 - | |—— DataDisp.msg



截至目前，主要阅读集中在代码框架的核心逻辑，以及核心的MINCO表示和优化（也就是 `traj_opt` 这一文件夹内的内容）

轨迹优化完整 workflow

整体上本代码库遵循了 mapping -> 前端路径搜索 -> 后端MINCO优化的流程。下面是相关的速查表：

论文模块	核心 C++ 类	文件路径	相关理解
建图	GridMap	plan_env -> grid_map.cpp	生成地图，获得是否
前端搜索	AStar	path_searching -> dyn_a_star.cpp	前端使用A*算法
轨迹表征	Piece/Trajectory	traj_opt -> poly_traj_utils.hpp	前者是单段表示，后者是整个MINCO轨迹。gcopter.cpp是
后端优化	PolyTrajOptimizer	traj_opt -> poly_traj_optimizer.cpp	使用L-BFGS算法
状态及管理	EGOReplanFSM	plan_manage -> ego_replan_fsm.cpp	状态机，决定何时Stop。这也是

从审稿看前沿：LLM + UAV Swarm

上周接到审稿任务：BCP-Swarm，其试图用LLM直接生成Swarm策略。

TABLE IV
 R_s OF n_{total} UAVS TRACKING SIX TARGETS IN THE SAME MAP.

Model	1 UAV	2 UAVs	4 UAVs	6 UAVs
ChatGPT-4o	80%	70%	100%	90%
ChatGPT-3.5	80%	90%	80%	80%
Qwen-2.5	50%	50%	60%	40%
DeepSeek-R1	80%	70%	70%	50%

- 本论文核心思路：约束LLM输出至原子指令（状态机转换），随后建立过滤器手动过滤不正确指令。本论文对比的是纯启发方法，成功率实际上受到LLM自身的空间推理关系影响巨大。本论文方法上很superficial，揭露了直接应用LLM的局限性，只安全但不聪明

以此论文为起点，调研了数篇文章，可以看出目前的主流调研均还在一个很初级的范畴之内

论文名称简写	特点速查	面向规模
BCP-Swarm	约束LLM输出至原子指令（状态机转换），随后建立过滤器手动过滤不正确指令	小规模(6架以内)
SwarmGPT	LLM基于音乐节拍，生成无人机waypoint和运动primitive，分布式优化控制	中等规模(~20架)
FlockGPT	基于用户需求生成符号距离函数，获得无人机与目标表面的距离，随后通过L-BFGS算法让无人机均匀分布	大规模(>100架)
UAV-VLA	根据自然语言解析指令生成目标，搜索物体，转换为MAVProxy的标准飞行指令序列。	单机，可扩展
AeroAgent	使用ROSCain将原始ROS观测转换格式，建议记忆库后在特定动作库内生成原子操作序列后，MPC跟踪后转换回ROS控制序列	单机，可扩展

下周计划：MINCO和代码库更深入钻研

下周具体主要处理下面的任务：

- 深入推导MINCO的梯度传导、以及优化的全流程
- 开始结合状态机文件，拆解无人机规划和决策整个流程
- 继续调研LLM与集群的结合（支线）。

预计至下周，可以初步而全面地形成对于EGO代码库的认知，从week 3开始，将更加深入，并逐步迁移到应用。

长期计划讨论：从仿真开始

本代码在仿真上采取了一个简单的实现。其具备了仿真完整流程：深度相机（`pointcloud_render_node.cpp`）模拟和对应的点云获取（`pcl_render_node.cpp`）、四旋翼（`Quadrotor.cpp`，初步模拟了推进器等，在SO(3)空间上仿真）。尽管如此，其不具备很多实际部署流程所存在的问题，例如其没有任何观测噪声（只涉及到理论层级的数学坐标系变换）、没有底层飞控等系统层级的模拟，与实际部署相去甚远。

考虑到现在存在一些相对更加完善的仿真，尽管其仍不能替代实机部署，但可一定程度上替代API/噪声特性等。在此想听老师看法和后续建议（预计将在week 3-4逐步迁移）：

- 继续沿用现有仿真平台。**是否需要后续更换仿真？**
- 无人机传统仿真典型路径是利用物理仿真引擎（XTDrone和Prometheus使用的是Gazebo，Airsim使用的是虚幻引擎），都支持PX4 SITL在环仿真（部分支持硬件在环），部分支持MAVLink。**生态成熟，并行支持差**

- 当前具身派使用Isaac等现代的仿真平台，例如Pegasus。**并行化优势，生态不成熟，需接口适配**

TABLE I

COMPARISON OF THE FEATURES PROVIDED BY MULTIPLE OPEN-SOURCE SIMULATORS AND MULTIROTOR SIMULATION FRAMEWORKS

Simulation Framework	Base Simulator/ Rendering Engine	Photo-realistic	Onboard sensors (IMU, GPS,...)	Vision Sensors	MAVLink Interface	API Complexity
jMAVSim	Java	x	✓	x	✓	*
RotorS	Gazebo	x	✓	✓	✓	**
PX4-SITL	Gazebo	x	✓	✓	✓	**
AirSim	Unreal Engine (or Unity)	✓	✓	✓	✓	***
Flightmare	Unity	✓	✓	✓	x	***
MuJoCo	OpenGL (or Unity)	x	x	x	x	**
Pegasus	NVIDIA Isaac Sim	✓	✓	✓	✓	**

✓ Indicates the presence and x the absence of features.
* Indicates the degree complexity of the different frameworks, on a scale from one to three stars.

附录：文献简要对比和速查（上周）

高飞老师

论文名称简写	特点速查
Primitive-Swarm	极大规模无人机调度，空间换时间
EGO-Planner系列	MINCO，集群，最终至Scirob
Robust traj planning for formation	基于拉普拉斯图矩阵作为MINCO优化目标，简化优化目标，编队整体形状变化协调
Sparse Graph	前一篇论文的改进，稀疏矩阵，降低计算复杂度
Aerobatic Flight (RL)	RL sim2real端到端，稀疏奖励+自适应课程学习，域随机化
Unlocking aerobatic potential	无人机特技飞行，偏航补偿映射（速度矢量空间），MINCO
USS-Nav	最新作，典型的NAV系列工作，LLM使用克制，仅用于推理目标区域，轻量级规划
Hand-like UAV grasping	一方面机械设计美感，另一方面通过白盒传感器估计和动力学克服质心等变化

其他文献（第一次）

论文名称简写	特点速查
D-Map - Fu Zhang	栅格图基础建立，直接基于点云标记+八叉树快速点云转栅格地图
Swarm-LIO2 - Fu Zhang	多机SLAM，LiDAR特性直接识别其他UAV，因子图，广播和退化机制
Proximity-Aware (ICCV '23)	建立策略感知（预测作为表征学习）的社交导航基础
FALCON - Junwei Liang	在上一篇文章的基础上引入未来轨迹预测，相对于上一工作产生本质提升
IROS2025 Challenge	相对上一篇文章引入连续的风险数值预测，提供平滑梯度指引
ASCENT - Junwei Liang (未放上)	建立鸟瞰图，拓扑建模，粗粒度评估，细粒度LLM决策（楼层内/间），楼层导航

其他文献（第二次）

论文名称简写	特点速查
RMA (sim2real圣经)	基于历史状态生成隐式表达，策略具备“记忆”，符合快（执行）慢（记忆）系统
Learning on the fly	简单模型，残差网络，在线训练，可微直接优化
RoboBallet	RL+GNN，从而将规划转移到离线训练，图表示任务关系，使用HER重放+单独惩罚
NIPS2025 MARS Challenge	作为对比，规划和控制赛道都表明VLM直接决策仍局限于双机简单任务

2026-02-05(文献综述第二部分)

自我介绍

谢冠文 2020-2024 浙江大学海洋学院本科 2024-2027 清华大学深研院电子信息

重点文献1 - EGO-Swarm和纵向观察

2022年，高飞老师课题组重磅发布一篇Science Robotics，其核心主题是小型UAV集群在没有任何先验地图的情况下实现快速穿梭。但是从研究轨迹来看，这篇文章并不是孤立的，而有其显著的演进流程。下面是里程碑的四篇文献：

Fast Planner（2019）：面向单个无人机，快速飞行的规划框架，是后续两篇文章的基础。其结构遵循经典的“前端搜索 + 后端优化”范式。**前端路径搜索**用Hybrid-State A*算法在体素地图进行，采用运动基元（典型的空换时间），基于LQR生成满足动力学模型约束，解析扩展搜索。**后端优化**根据前端的粗糙路径优化，先用均匀B样条优化轨迹，随后通过时间优化，将B样条从均匀转化为非均匀，将超过约束加速度等的轨迹端时间拉伸。强调轨迹质量（激进、时间最优）和理论完备性。

EGO-Planner（2020）：本文核心是克服Fast Planner对于ESDF强依赖的问题。某一段轨迹穿入障碍物时，系统会以该段为起点和终点，用 A* 算法快速搜索一条初步的局部无碰撞路径 Γ ，随后对穿障的每个控制点，从 Γ 上找一个对应的“锚点”，并构建一个排斥方向向量（从锚点指向障碍物表面）。其事实上就是在“线”这个维度上与本来的ESDF梯度形成了替代关系（从查询点指向最近障碍物表面的单位向量。因为只要方向是安全的，优化就能收敛。所谓锚点、排斥方向向量 $\{p, v\}$ 对）。

EGO-Swarm（2020）：本文核心是验证一个去中心化多机系统在未知杂乱环境中的可行性。重点是解决互避撞、通信、定位漂移等基础问题。基于EGO-Planner的改进，本文引入了三个核心创新点：

- 在原始 $\{p, v\}$ 对，生成一个对称的 $\{p_{new}, -v\}$ 对。这提供了多个搜索起点。
- 每架无人机独立规划自己的轨迹，根据广播轨迹优化目标函数中加入一个群体碰撞惩罚项，对互相接近施加软惩罚（即小于安全距离，随着穿透深度平方增长，确保梯度连续）。安全距离进行椭球变换，从而为z方向要求更大的距离，避免下洗气流的互相影响。

- 为了避免VIO的累积相对定位漂移，可以直接从深度图中检测到其他无人机的位置（检测到无人机对应3D点云的质心。本文还引入了额外判据提升鲁棒性），以及接收到的轨迹位置进行对比，这样通过低通滤波器直接输出相对漂移作为补偿。需要注意：作者说明本方法依赖于无人机可见，以及轨迹跟踪误差很小。

最终，EGO-Swarm 的前端拓扑搜索快两个数量级，空旷环境飞行达成SOTA效果。

Science Robotics（2022）：最终版本，核心在于提出一个面向真实世界部署的微型无人机集群平台，强调 TEEM 原则：轨迹最优性、可扩展性、经济计算、微型化。下面是主要变化：

- 采用MINCO轨迹优化：原始的均匀B样条仅优化轨迹的空间形状，时间分配是固定的（非均匀B样条可以调整时间分配，但是强耦合，非线性）。这可能导致在狭窄通道中，后方的无人机会“绕远路”等待前方无人机通过，产生次优甚至不安全的轨迹。MINCO的时空联合优化允许无人机可以通过平滑地调整速度来协调通过狭窄缝隙。
- 本文是软件和硬件结合的系统性工程。使用无人机重量不足300克，包含深度相机构建局部栅格地图（同时从深度图剔除掉无人机像素。这点在EGO-Swarm也进行了），且包含了UWB模块从而测量其他无人机相对距离（校正VIO漂移，这也是EGO-Swarm所没有的）。
- 本文可以被sci. rob录用的最核心因素是**验证了无人机在野外极端环境应用的潜力**，真正的集大成之作。本文达成了统一多目标优化框架，包含编队飞行、多机跟踪和密集互避评估三个任务。

Fast/EGO/MINCO三者对于避障的对比

- Fast Planner是对于障碍物避免常规的做法，因为障碍物距离的梯度需要ESDF给出，时间上消耗显著。（梯度指引需要连续可微距离场，其需要依赖于occupancy grid等多步传播计算，且增量更新复杂）
- EGO本质上是用一种方法，不使用ESDF去“估计”梯度：生成绕行路径，提取安全向量，定义锚点，计算投影距离。换言之，其使用局部几何启发式替代了全局的精确距离场计算。
- MINCO直接将避障作为硬约束（即：安全飞行走廊。需要注意，这一步仍然需要前端生成），通过微分同胚映射转化为了无约束问题，源头上避免碰撞，达成了数学一致性。

这一演进不仅是算法层面的迭代，更是对“真实世界部署”这一终极目标的不断逼近。在真实世界（尤其是野外、水下等非结构化环境）中，感知永远是局部的、带噪声的、动态变化的。系统设计必须以“不信任全局地图”为前提，转而强化在线重规划能力和对局部信息的高效利用。

重点文献2：Primitive Swarm

尽管基于MINCO和EGO的方法已经体现了强大的集群调度功能，但其对于数百上千的无人机集群，仍然偏重（优化项变多）。相比于前面的文章，本文的重点集中在：如何在超大规模的无人机集群下，满足强实时性需求的无人机调度。本文最核心的思想就是“用空间换时间”，处处体现工程智慧。

具体来说，本文用两步建立了时间最优运动基元库（Time-Optimal Motion Primitive Library）。第一步：本文建立了，从基础的圆弧几何单元出发，通过半径、三维空间旋转扩展等两个角度，扩展出初步的圆弧轨迹群。第二步，本文从轨迹推算出每个位置对应的时间，满足动力学约束条件下时间最短。简单来说分为四步：离散化路径（分成N段，例如N=1000），建立动力学约束（最大速度约

束），从终点开始计算可控集（可行速度，停下来的合理速度计算），接下来从起点和速度直接递推。为了考虑控制连续的问题，本文构造了多个起始速度的基元。实际使用中，由于基元较短（3~6m），这种小速度不连续可通过底层控制器补偿。

最终，本方法形成了约180条轨迹片段，只需要38.6秒的离线生成，占用75M的存储空间。

随后是路径规划与避障。本文在避障并不通过采样点逐个轨迹查询障碍物，而是采用“索引”的思想：通过网格空间内计算轨迹的空间占用关系/时空占用关系，利用在线检测的方式，这样某个网格有障碍物可以立刻标记相关空间占用关系内所有轨迹为不安全。若网格会被其他无人机占据，则检查时间是否冲突。这样的查表时间与基元的数量无关，只与障碍点云数量有关。

最后是轨迹选择与局部重规划。在网格优化的背景下，最终的优化仅仅考虑：轨迹离目标更近，轨迹没有出边界。每架无人机在自己的速度对齐坐标系中进行选择，保证轨迹衔接平滑。从安全基元中选中成本最低者执行。采用滚动时域策略，感知范围内不断局部重规划，异步触发。

最终，本文在 150 个障碍 + 80 架无人机的极端场景下仍保持>95%成功率；并且成功仿真 1000 架无人机在未知城市环境（高楼林立）中交换位置。总的来说，本论文并没有追求完整性能最优和长时的性能优化，而是在保证基本的轨迹合理性前提下，追求极致的算法效率。“离线预计算 + 在线轻推理”是大规模系统的必由之路。

Primitive-Swarm 成功的关键在于：轨迹生成与碰撞检测解耦；机器人间仅共享轨迹，不共享优化状态；无全局同步。MINCO-Swarm虽然相对较重（依赖于地图），但是轨迹平滑、控制连续、理论完备。

Robust and Efficient Trajectory Planning for Formation Flight in Dense Environments: 本文适用于障碍物林立的无人机编队，目标是：平衡编队保持和避障冲突，编队通过狭窄通道和空洞，编队指令变化快速响应。因此本文确立了使用全连接图作为编队建模基础（边权重是两机间距离的平方），通过计算该图的归一化拉普拉斯矩阵，来描述队形的几何结构。

为了加速大规模编队的时空优化（直接优化矩阵具平方复杂度），先离线计算出每架无人机为达到最佳队形所应处的“最优队形位置序列”，随后直接用二次惩罚替代相似度计算。在MINCO层级上，优化目标就是二次距离惩罚、动态可行性、障碍物避碰及机间互避。集群间的时空耦合是一个很大的问题，因此本文直接用“固定时间间隔采样”结合MINCO联合优化。

为了实现编队变化下的协调，本文提出了集群重组与共识机制。本文第一引入队形对齐优化，借助量化的队形约束感知度，最小二乘优化队形的缩放因子与平移向量，主动适应孔洞、走廊等受限空间；任务重分配优化即编队变化/重组时为每一个UAV分配目标点。本问题NP-hard，但仍然可以通过二分图最小权匹配，并且通过构造伪代价使得最优分配与缩放因子/平移无关。

总而言之，集群优化的最核心仍是明确可优化的指标，以及低的时空复杂度。

Sparse-Graph-Enabled Formation Planning for Large-Scale Aerial Swarms: 前文基础的改进。本文章是基于图模型，拉普拉斯矩阵定义编队约束的背景下，发现随着无人机数目增加，计算复杂度呈平方级别增长。因此本文思考在保证图全局刚性情况下保持完全图相近的编队精度和恢复能力。作者提出了选择至少4个非共面基点，随后其他无人机只与基点连接的构造机制，全局刚性可以被严格证明。全局刚性仅是基本保证，基点的选择极大影响编队性能。由于本问题NP-hard，作者使用遗

传算法，并且直接用矩阵的迹（提出的四种矩阵揭示度量中的一种）作为优化目标（计算快，三次复杂度降低为一次）。

Hand-like autonomous flying robot for airborne grasping and interaction: 本文探索了飞行+抓取的全新范式。本文结构上设计简单，一根尼龙绳通过滑轮系统，由一个伺服电机驱动整个手部变形，用最少的驱动器实现最大灵活性；2个扭转关节+3个伸缩模块，可自适应不同形状物体。最终整体尺寸仅556克。

本文伺服电机角度与手部形态的二次映射模型，快速生成抓取动作。本文抓取之后，最大的挑战在于模型参数变化（如重心、惯量随抓取改变），因此需要在线估计COG和惯量。本文也没有采取计算估计，而是直接映射（伺服电机角度和绳索位移推导当前各模块几何构型）。

本文另一个关键是扰动控制。本文实现上分为位置、姿态、抓取三个环。其中位置控制借助动力学公式，状态预测器在线估计，只对匹配扰动（沿机体Z轴，可被推力抵消）前馈补偿，非匹配扰动间接抑制；逆姿态控制直接用传感器测量值，通过增量修正期望角加速度来抵消扰动。这些内容最终被合并到了单个控制公式中。

总之本文使用的扰动估计方法很少，而是底层控制器和完整的传感器辨识来对抗扰动。HI-ARM的多级自适应策略并非单一算法堆砌，而是一个面向具身交互的协同控制体系：它将生物启发的硬件可变性与现代自适应控制理论深度融合，有效解决了“飞行-操作”强耦合带来的稳定性难题，为紧凑型空中操作机器人提供了可复用的控制范式。

Learning on the Fly: Rapid Policy Adaptation via Differentiable Simulation: 本文是Davide Scaramuzza课题组的全新力作（RAL2026）。本文认为DR不能覆盖所有可能的现实扰动，尤其对OOD扰动（如突然加装一个不规则负载）效果差；Real2Sim2Real需要大量真实数据+昂贵的离线重训练。基于此，本文在飞行时直接在线学习。具体来说，本文采用无人机在线与电脑连接，随后训练残差策略对仿真和现实的差异实时训练策略补偿。同时，本文将“简单模型+残差网络”组合成一个可微分的混合动力学模型，直接优化策略，而不是类似于PPO试错采样。最终，本文实时收集轨迹数据，直接更新残差模型，随后直接在可谓仿真器调整策略，在4090显卡上每5秒回传一次策略。需要注意，本文主要的聚焦在于域外扰动，尤其是 2m/s^2 的超大加速度干扰（OOD）。

重点文献3 - RoboBallet: Planning for multirobot reaching with graph neural networks and reinforcement learning

本文是2025年9月scirob的封面文章，旨在解决高密度多机器人协同作业中的任务分配、调度与无碰撞运动规划这一长期存在的工业难题。

在现代智能制造（如焊接、装配、喷涂）中，常需多个机械臂在共享、障碍物密集的工作空间内协同完成大量任务。理想情况下，增加机器人数量可提升吞吐量，但随之而来的是组合爆炸式复杂度：任务分配（谁做什么任务）、任务调度（安排顺序）、运动规划。

因此，作者提出一个因此，作者提出一个端到端可学习的自动化规划框架RoboBallet，以实现大规模、零样本泛化、实时响应的多机器人协同规划。

本文的核心在于：用强化学习+图神经网络将“人类直觉”自动化，把昂贵的在线规划转移到离线训练阶段。本文使用“图”表示协调关系：用机器人、任务（末端位姿）和障碍物作为节点，机器人之间（双向，避碰协调），任务/障碍物对机器人（单向，提供环境信息），从而支持关系推理。

节点

- **机器人节点**：7维关节角、7维关节速度、剩余任务停留时间（dwell time）。
- **任务节点**：仅含一个二值状态（是否已完成）。
- **障碍物节点**：无节点特征，仅通过边传递几何信息。

边

- **机器人 ↔ 机器人**（双向）：用于协调与避碰，边特征为相对末端位姿。
- **任务 → 机器人**（单向）：任务目标位姿相对于机器人末端的6D表示（3D平移 + 6D旋转）。
- **障碍物 → 机器人**（单向）：障碍物中心点、朝向、尺寸（以末端坐标系表示）。

随后这个架构使用Graph Nets（Battaglia et al., 2018）作为GNN核心架构，分别边更新、节点更新和全局更新。这样的GNN架构设计使得模型复杂度与机器人/任务数量无关，计算时间对机器人成二次复杂度。

另外一个关键难点是稀疏奖励。对于多达40个任务的稀疏奖励场景，本文几乎全部使用HER（Hindsight Experience Replay）重放，没有人工reward shaping。这使得agent能从任意轨迹中学习“如何到达某位置”，而非仅从成功episode中学习。任务奖励稀疏，但是障碍也单独施加了惩罚，仿真中硬性阻止碰撞（关节速度导致碰撞，速度直接设置为0），双重措施提升安全性。

最终，本文可以处理8台机器人、40个任务、56维联合构型空间，远超现有方法（此前SOTA仅支持5机10任务）。总而言之，本文提供了一种可扩展、可泛化、可语义增强的多智能体协同框架，是所谓“语义集群”的关键技术指引。

横向观察：从NIPS2025 MARS Challenge开始

自然语言背景下集群的语义调度也开始为学术界所关注。本部分将深入解读2026年1月26日新鲜出炉的技术报告。这一竞赛基于VIKI-Bench构建，分为两个赛道：Planning和Control，其中前者要求给定自然语言指令（如“把所有食物放进冰箱”）和场景图像，系统需选择机器人参与，并设计协作计划（如谁先开门、谁拿面包等）；后者需要在Maniskill环境中让2~4个机械臂协同完成操作任务。本文详细列举了两个赛道的冠军、亚军解决方案，从而给出洞察。

其中规划赛道的冠军（EfficientAI）分为三个阶段：

- 起点是少量人工标注的种子任务，基于Gemini 2.5根据这些种子任务的指令和场景，创造性地生成大量新的、多样化的训练数据。例如，原始任务是“打开冰箱”，模型可以自动生成补充任务“把所有食物放进冰箱”。
- 使用上述数据做SFT训练，对于新的测试任务，多次推理生成多个候选计划，引入一个“裁判VLM”对这些候选计划进行评估和打分，或者直接采用投票机制，选择出现频率最高的动作作为最终决策。

- 将经过“裁判”筛选出的高质量计划，再次加入到训练数据集中，用这个更优质的混合数据集重新训练模型。如此循环往复，模型和数据集的质量都得到持续提升，而无需大量的人工标注。

亚军方案（TrustPath）则认为，一个复杂的多智能体规划任务可以被清晰地分解为几个独立但又相互协作的子任务，并由专门的模块来处理。这种方法可解释性强，能有效减少幻觉。

- 激活模块接收任务指令和场景图像，结合预定义的机器人能力先验知识（例如，Fetch机械臂擅长抓取，Stompy人形机器人擅长开门），来决定哪些机器人应该参与本次任务。
- 规划模块在激活模块选定的机器人集合基础上，生成详细、并行、且符合物理约束的动作序列。需要注意，这个动作序列是高层的（例如：`{"R1": ["Reach", "apple"], "R2": ["Open", "cabinet"]}`）
- 监控模块从语法有效、能力对齐（能力是否匹配）、逻辑可行三者评判可行性并修正。
- 需要注意，前两个模块都是基于VIKI数据集做SFT训练的。

二者最终得分接近（前者0.893，后者0.858），且都远远高于第三名（0.722）。二者代表了截然不同的LLM规划策略（前者黑箱，随机，计算昂贵，针对模糊任务鲁棒性相对较好；后者白盒，确定，计算成本较低，依赖先验数据集）。

从更加整体的情况来看：简单任务（例如打开烤箱）多数团队都可以完成，但是困难任务（例如“把所有食物放进冰箱”）平均得分仅有0.16，这一方面是未识别全“食物”包含哪些物体（指令模糊），让机器人轮流干活，效率低（“缺乏并行”），第一步选错机器人，后面全错（早期错误放大）。另外，论文中提到存在“Lazy Plan”问题，即VLM模型倾向于执行“覆盖所有可能性”的冗余动作（如把所有水果都放进碗），而非精准推理最小动作序列。这说明：纯 prompt 调用开源 VLM 无法满足规划赛道对效率、精确性和并行性的要求。因此，两个 top 方案都选择了“微调专用模型”路径，只是架构哲学不同。

控制赛道的冠军（Combo-MoE）使用一个主干VLM，但是使用Mixture-of-Experts (MoE) 层作为动作头，每个expert对应机械臂组合（例如Arm1、Arm2+Arm3、Arm1+Arm2+Arm3+Arm4等，随智能体个数指数增长，例如4个智能体有15个头）。随后用激活权重进行融合最终解码输出。训练则分为三部分：专家预训练（每个expert单独训练），冻结专家后训练路由策略，整体微调。

控制赛道的亚军（CoVLA）则使用去中心化协作，不依赖中央控制器，而是让每个机械臂运行自己的VLA策略，通过共享视觉环境实现隐式通信与协调。换言之，每个策略完全预先分配任务，随后直接基于 π_0 模型微调。

尽管如此，最终的实验结果也说不上理想（前两个任务是双臂，后面分别为3、4臂）。归根到底，这仍然是因为动作空间爆炸、缺乏有效的协调机制、仿真到现实的泛化能力不足。

Table 2: Task success rates (out of 100 trials) of the top three teams in the Control Track.

Task	MMLab@HKUxD-Robotics	INSAIT	RoboStar	Average
Place Cube in Cup	57%	13%	15%	28.3%
Strike Cube (Hard)	12%	9%	7%	9.3%
Three Robots Place Shoes	0%	2%	1%	1.0%
Four Robots Stack Cube	0%	0%	0%	0.0%

当然，现在对于语义协作，范式远没有固定，在短期来说，VLM适合作为某一个中间的数据生成器或者决策器，但还远不足以对复杂长程的任务进行规划。

USS-Nav: Unified Spatio-Semantic Scene Graph for Lightweight UAV Zero-Shot Object

Navigation: 高飞课题组与北航合作，2026年2月3日最新力作。本文提出一种面向资源受限无人机的轻量级零样本物体导航框架。其核心创新在于构建统一的时空-语义场景图：通过多面体膨胀增量生成稀疏空间连通图以表征全局几何拓扑，并结合图聚类实现动态区域划分；同时利用轻量视觉模型（YOLO-E + MobileCLIP）在线实例化开放词汇语义物体，并锚定至拓扑结构，形成层次化环境表示。在此基础上，系统采用粗到细的分层决策策略——由大语言模型基于场景图语义推理目标区域，局部则通过TSP求解器优化前沿探索路径。该方法在 Jetson Orin NX 上实现 15 Hz 实时更新，显著优于现有方法，在复杂未知环境中 SPL（Success weighted by Path Length）提升达 146.61%，首次在真实无人机上实现高效、语义驱动的零样本导航。

计划与杂项

- 从SciRob和MINCO开始复现，手推MINCO：安装ubuntu 20.04 -> 三个项目复现 -> 阅读和手推
- 春节后可能需要长时间赴河北大厂出差（预计时间在1-2个月），主要目的是AUV设备的加装和水池测试，时间可能是开学后，也可能是3月后。届时计划再确定（sim2real? 采购?）